



BAŞKENT ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ
BİTİRME PROJESİ RAPORU

BAŞKENT ÜNİVERSİTESİ E-DOKÜMANTASYON VE DEĞERLENDİRME
SİSTEMİ(BEDES)
BAŞKENT ÜNİVERSİTESİ MEZUNİYET KOŞULU KONTROL UYGULAMASI
(BMNKU)

Arda ÖZAN
22196372
Onur Uygur KÜÇÜK
22296857
Yaşar Eren KAYGIN
22195940

Bölümü: Bilgisayar Mühendisliği
Proje Danışmanı: Doç. Dr. Çağatay Berke Erdaş
Ders Kodu ve Adı: BİL 493 Bitirme Projesi 1
Proje Başlangıcı: 2025/2026 Güz Proje Süresi (Yarıyıl): 2 / 2
Rapor Sunumu: 2025/2026 Güz

ONAY SAYFASI

Bu Rapor, / 01/ 2026 tarihinde ařağıda üye adları yazılı jüri tarafından kabul edilmiştir.

Unvan	Adı Soyadı	İmza
Doç. Dr.	Çağatay Berke ERDAŞ (Danışman)	
Prof. Dr.	Murat HACİÖMEROĞLU	
Doç Dr.	Hazel YÜCEL	
Dr. Öğr. Üyesi	Cansu BETİN ONUR	

ÖZ

Danışmanlık ve ders planlama süreçleri, birçok eğitim kurumunda hâlâ parçalı araçlar üzerinden yürütülmekte; ders seçimi, ön koşul uygunluğu, kapasite/çakışma kontrolü ve danışman-öğrenci iletişimi gibi kritik adımlar çoğu zaman manuel yöntemlerle takip edilmektedir. Bu durum hem öğrenciler hem akademik danışmanlar açısından zaman kaybına, hatalı ders kayıtlarına ve dönem içinde tekrar eden düzeltme işlemlerine yol açabilmektedir. Özellikle ders programı çakışmaları, kontenjan yönetimi ve öğrencinin mezuniyet koşullarına uygun ders seçebilmesi gibi konuların sağlıklı yönetilmemesi, eğitim planlamasında verimliliği düşüren temel problemler arasında yer almaktadır. Bu ihtiyaçlardan hareketle geliştirilen sistem, akademik süreçleri tek bir platformda toplayarak daha hızlı, güvenilir ve izlenebilir bir dijital ekosistem oluşturmayı amaçlamaktadır.

Geliştirilen proje, öğrenci, danışman ve yönetici rollerini bir araya getirerek uçtan uca bir ders ve danışmanlık yönetimi sunar. Öğrenciler, “ders programı belirleme” ekranı üzerinden derslerin hangi gün ve saatlerde yapıldığını detaylı biçimde görüntüleyerek uygun dersleri seçebilir. Sistem öğrencinin seçtiği derslerde zaman çakışması olup oluşmadığını otomatik olarak analiz ederek hatalı kayıtların önüne geçer. Bunun yanında derslerin kontenjan doluluk durumu takip edilir; kontenjanı dolu derslerde kayıt işlemi engellenerek kayıt süreci üzerindeki kontrol artırılır. Böylece ders seçimi, yalnızca bir listeleme işlemi olmaktan daha çok; kuralları ve akademik kısıtlamalara sahip güvenilir ve düzenli bir planlama sürecine dönüşür.

Akademik danışmanlar açısından sistem, danışmanına atanmış öğrencilerin durumunu tek panelde izleyebilme, ders kayıtlarını gözlemleyebilme ve gerekli durumlarda bilgilendirme sağlayabilme imkânı sunar. Bu sayede dönem içerisinde öğrenci tarafından gerçekleştirilecek kritik öneme sahip ders seçimi gibi işlemler danışman tarafından güncel olarak izlenebilir ve bildirim altyapısı sayesinde sürecin sağlıklı biçimde yönetilmesi amaçlanır. Yönetici rolü ise ders havuzu, ders oluşturma ve genel sistem yönetimi gibi operasyonları merkezi bir noktadan gözlemleyip yönetebilir.

Buna ek olarak, akademik danışmanların öğrencilerin ders geçmişlerine dayalı olarak ön koşul uygunluklarını hızlı ve hatasız biçimde değerlendirebilmeleri amacıyla, web sisteminden bağımsız çalışan Electron tabanlı bir masaüstü karar destek uygulaması geliştirilmiştir. Bu uygulama, öğrencinin geçmiş ders durumlarını analiz ederek her ders için “alabilir/alemez” sonucunu otomatik olarak üretmekte ve eksik ön koşulları açık biçimde göstermektedir.

Sistem mimarisi, modern web tabanlı geliştirme pratiklerine uygun şekilde tasarlanmıştır. Verilerin tutarlı biçimde depolanması, güvenli kimlik doğrulama ve rol bazlı yetkilendirme sağlanmıştır. API üzerinden standartlaştırılmış servis

erişimi gibi unsurlar temel alınmıştır. Bu yapı, kurumsal ölçekte sürdürülebilir bir geliştirme ve bakım süreci sağlar. Ayrıca dijital kayıt ve raporlama yaklaşımı sayesinde dönem sonunda ders seçimi ve devam eden diğer süreçler hakkında anlamlı veriler elde edilebilir. Bu raporlama altyapısı, sistemin ileriye dönük gelişimi ve hataların giderilmesi için hayati bir rol oynamaktadır.

Proje, Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları kapsamındaki 4 numaralı “Nitelikli Eğitim” hedefiyle ilişkilidir. Eğitim süreçlerinde erişilebilirliği artıran, hataları azaltan, öğrenci ve danışman arasındaki iletişim eksikliklerini en aza indirmeyi hedefleyen bu yaklaşım; eğitimde kaliteyi artırmaya yönelik dijital dönüşüm çalışmalarının bir parçasıdır.

ABSTRACT

Advising and course planning processes in many educational institutions are still carried out through fragmented tools; critical steps such as course selection, prerequisite eligibility, capacity/schedule conflict checks, and advisor–student communication are often tracked manually. This situation can lead to time loss for both students and academic advisors, incorrect course registrations, and recurring correction procedures throughout the semester. In particular, issues such as timetable clashes, quota (capacity) management, and ensuring that students can choose courses aligned with graduation requirements are among the main problems that reduce efficiency in educational planning when they are not managed properly. In response to these needs, the proposed system aims to bring academic processes together on a single platform, creating a faster, more reliable, and more traceable digital ecosystem.

The developed project provides end-to-end course and advising management by bringing together the roles of student, advisor, and administrator. Through the “course schedule planning” screen, students can view in detail on which days and at what times courses are held and select the courses that fit their availability. The system automatically analyzes whether the selected courses create any time conflicts, preventing incorrect registrations. In addition, course capacity status is tracked; registration is blocked for courses that have reached full capacity, increasing control over the enrollment process. As a result, course selection becomes more than a simple listing action—it turns into a reliable and well-structured planning process governed by rules and academic constraints.

From the perspective of academic advisors, the system enables them to monitor the status of the students assigned to them from a single dashboard, review course registrations, and provide guidance and notifications when necessary. In this way, critical student actions during the semester—such as course selection and approaching deadlines for required tasks—can be tracked in real time by the advisor, and timely warnings can be delivered to students through the notification infrastructure to ensure the process is managed effectively. The administrator role, on the other hand, can centrally oversee and manage operations such as the course catalog, course creation, and overall system administration.

In addition, an Electron-based standalone desktop decision support application has been developed to assist academic advisors in evaluating students’ prerequisite eligibility more efficiently. This application analyzes a student’s academic history and automatically determines whether specific courses can be taken based on predefined prerequisite rules, clearly indicating “eligible” or “not eligible” outcomes along with missing prerequisites when applicable.

The system architecture has been designed in accordance with modern web-based development practices. It ensures consistent data storage, secure authentication, and role-based authorization. Key principles include standardized service access through an API. This structure supports a sustainable development and maintenance process at an enterprise scale. In addition, thanks to the digital record-keeping and reporting approach, meaningful insights can be obtained at the end of the term regarding course selection and other ongoing processes. This reporting infrastructure plays a vital role in the system's future evolution and in identifying and resolving issues.

The project is aligned with the United Nations Sustainable Development Goals, specifically Goal 4: "Quality Education." By improving accessibility in educational processes, reducing errors, minimizing communication gaps or mistakes between students and advisors, and strengthening academic planning, this approach forms part of broader digital transformation efforts aimed at elevating the quality of education to higher levels.

İÇİNDEKİLER

ONAY SAYFASI	1
ÖZ	2
ABSTRACT	4
İÇİNDEKİLER	6
ŞEKİLLER DİZİNİ	7
1. GİRİŞ	9
1.1. Problem Tanımı	9
1.2. Sistem Gereksinimleri	10
1.2.1. Fonksiyonel Gereksinimleri	10
1.2.2. Fonksiyonel Olmayan Gereksinimleri	12
1.3. Kısıtlar	14
1.4. Başarı Ölçütleri	14
1.5. Görev Dağılımı	15
1.6. Sistemin Genel Yapısı	17
1.7. İş-Zaman Planı	26
1.8. Veritabanı Tabloları	27
2. Araçlar Ve Yöntemler	31
2.1 Arda'nın Görevleri	31
2.2 Onur'un Görevleri	33
2.3 Yaşar'ın Görevleri	34
3. Sonuçlar	37
3.1. Hedef Özeti	37
3.2. Hedeflerin Gerçekleştirilmesi	38
3.3. Testler	39
4. Arayüz Fotoğrafları Ve Özellikleri	41
5.KAYNAKÇA	61

ŞEKİLLER

Şekil No	Açıklama	Sayfa No
1.1.	Sistemin Genel Mimarisi	19
1.2.	Kullanıcı Roller ve Sisteme Erişim	21
1.3.	Frontend Katmanı	22
1.4.	Backend ve Database Katmanı	24
1.4.1.	BMNKU Sistem Yapısı	26
1.4.2.	İş-Zaman Planı(BMNKU)	26
1.4.3.	İş-Zaman Planı(BEDES)	27
1.5.	AspNetUsers Tablosu	27
1.6.	StudentProfiles	27
1.7.	CourseCategories	28
1.8.	Courses	28
1.9.	CourseSchedules	29
1.10.	StudentCourseSections	29
1.11.	Documents	29
1.12.	DocumentVersions	30
1.13.	Notifications	30
1.14.	Comments	30

1.15.	İlişki Tablosu	31
2.1.	Giriş Ekranı	41
2.2.	Kayıt Ekranı	42
2.3.	Dashboard Ekranı	43
2.4.	Navbar	44
2.5.	Belge Görüntüleme	45
2.6.	Belge Arama	46
2.7.	Ders Kataloğu	47
2.8.	Öğrenci Yönetim ve Görüntüleme	48
2.9.	Danışman Atama	49
2.10.	Profil Ekranı	50
2.11.	Bildirim	50
2.12.	Bildirim Yollama	51
2.13.	Teslim	52
2.14.	Teslim Takibi	54
2.15.	Ders Seçim	55
2.16.	Ders Programı	56
3.1.	Ana Ekran	57
3.2.	Öğrenci Görüntüleme	58
3.3.	Ders Ön Koşul Görüntüleme	59
3.4.	Pdf Yükleme	60

1.GİRİŞ

1.1. Problem Tanımı

BEDES

Ekibimiz tarafından gözlemlendiği ve deneyimlendiği kadarı ile eğitim kurumlarında kullanılmakta olan eğitim ve öğretim sürecinin yönetilmesi ile sorumlu sistemler çoğu zaman farklı parçalar halinde dağılmış bir biçimde yürütülmekte veya manuel yöntemlerle takip edilmektedir. Öğrencilerin dersleri seçerken derslerin hafta içerisinde hangi gün ve saatlerde olacağını net görememesi, zaman çakışmalarını dersi seçene kadar görememesi, kontenjan durumuna göre düzgün bir planlama yapamaması ve ön koşul/mezuniyet şartlarını sağlıklı görüntüleyememesi/değerlendirememesi; dönem içinde hatalı kayıtlar, ekle-bırak haftasındaki işlem yoğunluğu sebebiyle gözden kaçırılan ayrıntılar, hatalı kayıtlar vb. sonuçlara yol açmaktadır. Benzer şekilde akademik danışmanlar ve yöneticiler açısından da öğrencilerin ders seçimlerini kontrol etmek, önkoşul gibi belirleyici unsurları değerlendirmek ve süreci takip etmek ciddi zaman kaybı yaratmakta ve iletişimin farklı kanallardan yürütülmesi sebebiyle takip edilebilirliği ve süreç yönetimi zorlaşmaktadır.

Ders programı çakışmaları veya kontenjan eksikliği sebebi ile öğrencilerin ders kayıtlarını değiştirmek zorunda kalmaları, danışman onay süreçlerinin uzaması, dönem başında ders seçim haftasındaki yoğunluk sebebiyle bu sürecin daha da karmaşıklaşan bir hal alıp uzaması, sistemin içerisinde bulunan tüm bireylerin kullanıcı deneyimini kötü etkilemektedir. Mevcut sistem içerisindeki raporlama sistemine dair bir bilginiz bulunmadığından, raporlamanın da yeterli ayrıntıları içermediği veya hiç bulunmadığı varsayımında bulunarak bu karmaşıklığın giderilmesi için raporlama eklenmesi, iyileştirme sürecini pozitif etkilemesi yönünden önemlidir.

Bu projenin problem tanımı, ders program bilgilerini doğru ve erişilebilir şekilde sunan, öğrencilerin seçtiği derslerin mümkün olduğunca otomatik kurallarla kontrol edilerek öğrenciye düşen yükün azaltılarak ders programına eklenilebilmesi, danışmanlık ve yönetim tarafında izlenebilir bir yapı sunan bütünleşik bir sistem ihtiyacıdır. Amaç; ders seçimi ve danışmanlık sürecini tek merkez etrafında toplayarak hatalı ve uyumsuz kayıtları azaltmak, dönem başı ve sonundaki operasyonel yükü düşürmek ve bu süreçleri akademik olarak daha kontrollü şeffaf ve sürdürülebilir hale getirmektir.

BMNKU

Eğitim kurumlarında akademik danışmanlık sürecinin önemli bir parçasını, öğrencilerin ders geçmişine göre hangi dersleri alabileceğinin değerlendirilmesi oluşturmaktadır. Bu değerlendirme çoğu zaman öğrencinin transkriptinin manuel olarak incelenmesi, ön koşul ilişkilerinin tek tek kontrol edilmesi ve farklı kaynaklardan bilgi toplanması yoluyla gerçekleştirilmektedir. Özellikle yoğun kayıt dönemlerinde bu süreç, akademik danışmanlar açısından ciddi zaman kaybına ve hata riskine yol açmaktadır.

Mevcut web tabanlı sistemler, ders seçimi ve program çakışması gibi süreçleri yönetebilse de; öğrencinin ders geçmişine dayalı ön koşul uygunluğunu hızlı, net ve danışman odaklı biçimde değerlendiren bağımsız bir araç ihtiyacı devam etmektedir. Bu ihtiyacın karşılanamaması, öğrencilerin yanlış yönlendirilmesine, hatalı ders planlamalarına ve dönem içinde tekrar eden ders kayıt düzeltmelerine sebep olmaktadır.

Bu bağlamda Electron tabanlı masaüstü uygulamanın problem tanımı; akademik danışmanın, spesifik bir öğrencinin ders geçmişini esas alarak, hangi dersleri alıp alamayacağını ön koşul kurallarına göre otomatik ve hatasız biçimde görebileceği bir karar destek sisteminin eksikliğidir. Geliştirilen uygulama, bu eksikliği gidererek danışmanlık sürecini daha hızlı, güvenilir ve standart hale getirmeyi hedeflemektedir.

1.2. Sistem Gereksinimleri

Bu bölümde sistem ile ilgili fonksiyonel olan ve fonksiyonel olmayan gereksinimler belirtilmektedir.

1.2.1 Fonksiyonel Olan Gereksinimler

BEDES

- Sistem, kullanıcıların kimlik ve yetki doğrulamasını gerçekleştirerek yalnızca yetkili kullanıcıların sisteme erişmesini sağlayacak.
- Sistem kullanıcı rollerini kontrol ederek her rolün erişebileceği ekran ve işlemleri sınırlandırılabilir.
- Sistem derslerin detaylarını(akts,kredi,açıklama vb.) görüntüleyebilecek.
- Sistem ders programı ekranında dersleri gün-saat, şube gibi bilgiler ile birlikte listeleyebilecek.
- Sistem öğrencinin ders programı ekranından ders seçip kendi programına eklemesini sağlayabilecek.
- Sistem öğrenci ders ekleme işlemi sırasında zaman çakışması durumunu tespit ederek çakışan derslerin eklenmesini engelleyebilecek.
- Sistem ders seçiminde kontenjan takibi yapabilecek. Kontenjanı dolu olan şubelere kayıt yapılmasını engelleyebilecek.

- Sistem öğrencinin seçtiği dersleri daha sonra görüntüleyebilmesini ve gerekli durumlarda dersten çıkma işlemi yapılmasını sağlayabilecek.
- Sistem öğrencinin dönemlik ders programını günlere göre görüntüleyebilecek ve program verisini frontend'e uygun formatta sağlayabilecek.
- Sistem ders detaylarında ilgili ön koşul derslerini gösterebilecek ve yönetebilecek.
- Sistem, danışmanların kendilerine atanmış öğrencileri listeleyebilmesini ve öğrencilerin ders/akademik durumlarını görüntüleyebilmesini sağlayabilecek.
- Sistem, bildirim mekanizması ile kullanıcıların önemli süreçler hakkında bilgilendirilebilmesini sağlayabilecek.
- Sistem admin rolü için ders, kategori, program oluşturma güncelleme silme işlemlerini yönetebilecek.

BMNKU

- Sistem, masaüstü ortamda çalışabilen bağımsız bir Electron uygulaması olarak çalışabilmelidir.
- Sistem, danışmanın bir öğrenciyi seçmesine veya öğrencinin ders geçmişini sisteme tanımlamasına imkân sağlamalıdır.
- Sistem, öğrencinin geçmiş ders durumlarını (geçti / kaldı) görüntüleyebilmelidir.
- Sistem, ders—ön koşul ilişkilerini tanımlı kurallar üzerinden değerlendirebilmelidir.
- Sistem, her ders için “Alabilir / Alamaz” sonucunu otomatik olarak üretmelidir.
- Ön koşulu sağlanmayan derslerde, eksik ön koşul dersleri kullanıcıya açık ve anlaşılır biçimde listeleyebilmelidir.
- Sistem, sorgulama sonucunu kullanıcı arayüzünde anlık olarak güncelleyebilmelidir.
- Kullanıcı arayüzü, danışmanın hızlı karar verebilmesini destekleyecek sade ve anlaşılır bir yapıda olmalıdır.

1.2.2 Fonksiyonel Olmayan Gereksinimler

BEDES

Proje tasarlanırken bütün sistemin Microsoft Azure ile entegre olarak çalışması planlanıldığından gereksinimler söz konusu kısıt göz önünde bulundurularak yazılmıştır.

- Sistem, tipik API uç noktalarında ortalama yanıt süresi $\leq 700ms$, P95 $\leq 2000ms$ olacak şekilde hedeflenecektir.
- Kayıt/çıkarma gibi yazma işlemlerinde ortalama yanıt süresi $\leq 1200ms$, P95 $\leq 3000ms$ olacak şekilde hedeflenecektir.
- Yoğun dönem başlangıcı senaryosunda sistem, en az 300 eşzamanlı kullanıcıyı karşılayabilecek, bu yük altında 5xx(sunucu tarafı) hata oranı $\leq \%1$ olarak hedeflenecektir.
- Sistem, Azure SQL üzerinde ders sorgularında P95 sorgu süresi $\leq 500ms$ hedefiyle indeksleme ve sorgu optimizasyonu uygulayacaktır.

a) Kullanılabilirlik

- Sistem aylık bazında en az $\%99.5$ erişilebilirlik hedefiyle çalışacaktır.(planlı bakımlar hariçtir)
- Kritik servis hatalarında(APp Service restart vb.)sistemin tekrar hizmete dönme süresi MTTR ≤ 10 dakika hedeflenecektir

b) Ölçeklenebilirlik

- Sistem Azure App Service üzerinde yatay ölçekleme ile büyüyecek şekilde tasarlanacaktır.
- Trafik artışında otomatik ölçekleme tetikleyici hedefleri
CPU $\geq \%70$ 5 dakikadan fazla sürerse instance arttırılacak.
CPU $\leq \%30$ 10 dakikadan fazla sürerse instance azaltılacak

c) Güvenlik

- Tüm istekler HTTPS olacak.
- Kimlik doğrulama JWT ile yapılacak;Token geçerlilik süresi 24 saat olacak şekilde yönetilecek.
- Sistem Azure SQL bağlantılarında Encrypt=True ve sertifika doğrulama politikalarıyla güvenli bağlantı kurulacak.
- Yetkisiz erişim denemelerinde 401/403 döndürülecek, hata mesajları iç sistem detaylarını sızdırmayacak.
- Veritabanı erişiminde en az ayrıcalık prensibi uygulanacak.

d) Veri Bütünlüğü ve Tutarlılık

- Ders ekleme ve çıkarma işlemleri transaction mantığıyla çalışacak. Yarım kayıt oluşması $\%0$ hedefiyle engellenecek.
- Aynı öğrencinin aynı ders veya şubeye tekrarlı kayıt girişimleri sistem tarafından engellenecek, tekrarlayan kayıt $\%0$ hedefiyle tutulacak.

-Kapasite aşırımları eş zamanlı kayıt denemelerinde dahi engellenecek

e) Loglama

-Uygulama logları Azure'da merkezi toplanacak şekilde yapılandırılacak.(Log Analytics)

-Kritik işlemler için log kaydı tutulacak ve olaylar en az 30 gün saklanabilir olacak.

-Uygulama hataları için uyarı hedefi: 5 dakika içinde 10 dan fazla 5xx hatası görülürse alarm verilecek.

BMNKU

a) Performans

-Ön koşul uygunluk sorgulama işlemi kullanıcı etkileşiminden sonra en geç 1 saniye içerisinde sonuç üretmelidir.

-Öğrenci ve ders verileri bellekte yönetilerek tekrar eden sorgulamalarda gecikme yaşanmamalıdır.

b) Kullanılabilirlik

-Uygulama, teknik bilgi gerektirmeden danışmanlar tarafından kolayca kullanılabilir olmalıdır. -Uygulama arayüzü, masaüstü ekran çözünürlüklerine uyumlu olacak şekilde tasarlanmalıdır.

c) Taşınabilirlik

-Uygulama Windows işletim sistemi üzerinde çalışabilir olmalıdır.

-Tek bir paket dosyası (.exe) ile kurulabilir veya çalıştırılabilir olmalıdır.

d) Güvenilirlik

-Yanlış veya eksik veri girişlerinde sistem kullanıcıyı bilgilendirmeli, uygulama çökmeden çalışmaya devam etmelidir.

-Ön koşul değerlendirme kuralları deterministik olmalı; aynı veri için her zaman aynı sonuç üretilmelidir.

1.3. Kısıtlamalar

- Veri Güvenliği ve Gizlilik: Kullanıcıya ait kişisel veriler KVKK kapsamında değerlendirilerek işlenecek ve gereksiz veri toplanmayacaktır.
- Yetkilendirme ve Doğrulama: Sistem rol bazlı erişim kontrolü uygulamaktadır. Bu sayede yetkisiz erişimler engellenir ve kritik işlemler yalnızca yetkililer tarafından yapılabilir.
- Bulut Ortamı Kısıtı: Sistem Azure üzerinde yayınlanacağı için servislerin sürekliliği, bağlantı güvenliği ve yapılandırma yönetimi Azure servislerine bağlıdır.
- Sürekli İnternet Bağlantısı: Uygulama Azure üzerinde çalıştığından istemcilerin internet bağlantısı zorunludur.
- Veritabanı Bağımlılığı: Ders programı, ders seçim işlemleri vb. işlemler veritabanı erişimine bağlıdır. Veritabanı erişiminde yaşanacak bir aksaklık sistemin tamamını etkileyecektir.
- Eşzamanlı Kullanım: Yoğun akademik dönemlerde çok sayıda öğrencinin etkileşimiyle oluşabilecek çift kayıt ve kontenjan aşımı gibi sorunlar engellenmek zorundadır.

Platform ve dağıtım kısıtı: Uygulama Electron ile çoklu platformu destekleyecek şekilde geliştirilebilse de, teslim sürecinde test/dağıtım belirli işletim sistemi(leri) ile sınırlı olabilir. Paketleme (build) ve kurulum dosyaları her platform için ayrı süreç gerektirebilir.

1.4. Başarı Ölçütleri

- Ders listeleme (schedule dahil) performansı: GET /api/course-selection/available isteğinin P95 yanıt süresi 2 saniyenin altında olmalıdır.
- Ders programı görüntüleme performansı: GET /api/student-courses/my-schedule isteğinin P95 yanıt süresi 2 saniyenin altında olmalıdır.
- Ders ekleme işlemi süresi: POST /api/course-selection/enroll işlemi (kontenjan + çakışma kontrolü dahil) P95 3 saniyenin altında tamamlanmalıdır.
- Veri tutarlılığı:
- Aynı öğrencinin aynı derse birden fazla kaydı %0 olmalıdır (çift kayıt tamamen engellenmelidir).
- Kontenjanı dolu şubeye kayıt denemelerinin engellenme doğruluğu %100 olmalıdır.
- Zaman çakışması olan derslerin eklenmesini engelleme doğruluğu %100 olmalıdır.

-Erişilebilirlik (Azure): Sistem aylık bazda %99,5 veya üzeri uptime sağlayabilmelidir.

-Hata oranı: Yoğun kullanım senaryosunda sunucu kaynaklı hata oranı (5xx) %1'in altında tutulmalıdır.

-Kullanılabilirlik: Öğrenci ve danışman kullanıcılarının en az %90'ı ders seçimi ve ders programı ekranlarını "kolay anlaşılır ve kullanılabilir" olarak değerlendirmelidir (anket/geri bildirim ile ölçülebilir).

-Veri uyumu: Ders kayıtları ile schedule verileri arasında uyumsuzluk (örn. kayıt var ama schedule yok gibi) oluşma oranı %0 hedeflenmelidir.

-Ön koşul uygunluk kontrolü doğruluğu: Ön koşulu sağlanan dersler için "Alabilir", sağlanmayanlar için "Alamaz" çıktısının doğruluğu %100 olmalıdır.

-Eksik ön koşul listeleme doğruluğu: "Alamaz" durumunda listelenen eksik ön koşullar, tanımlı kurallarla birebir örtüşmelidir.

Performans: Ders uygunluk sorgusu sonucu 1 saniyenin altında üretilmelidir.

Kullanıcı memnuniyeti: Danışman kullanıcıların en az %90'ı uygulamayı "hızlı ve anlaşılır" olarak değerlendirmelidir.

Kararlılık: Demo ve test senaryolarında uygulama çökmesi veya kilitlemesi yaşanmamalıdır.

1.5. Görev Dağılımı

1) Arda – Web Öğrenci Yönetim Sistemi (ASP.NET + React/Vite + Azure)

Arda, web tabanlı Öğrenci Yönetim Sistemi'nin geliştirilmesinden sorumludur. Bu kapsamda:

- **Veri modeli ve veritabanı (Azure) tasarımı** (öğrenci bilgileri, kayıtlar vb.)
- **ASP.NET API** geliştirme (CRUD işlemleri: ekleme/silme/güncelleme/listeleme)
- **React/Vite arayüzü:** yönetim ekranları (listeleme, detay görüntüleme, form ekranları)
- API–Frontend entegrasyonu, doğrulamalar ve hata yönetimi
- Testler ve kararlılık iyileştirmeleri

2) Yaşar – Electron Ön Koşul Kontrol Uygulaması (Electron + React/Vite)

Yaşar, masaüstü uygulamanın altyapı ve ana geliştirme süreçlerini üstlenmiştir. Bu kapsamda:

- Electron proje kurulumu, **main/renderers yapısı** ve uygulama mimarisinin oluşturulması
- Öğrenci ders durumlarının uygulama içinde **hafızada yönetilmesi** (state yönetimi / veri yapıları)
- Ön koşul kurallarını temsil edecek yapıların tasarlanması (ders–ön koşul ilişkileri)
- Uygulama akışı, temel ekranlar ve yönlendirmeler
- Build/paketleme ve hata ayıklama süreçleri

3) Onur – Electron Ön Koşul Kontrol Uygulaması (Electron + React/Vite)

Onur, masaüstü uygulamanın iş mantığı ve kullanıcı arayüzü tarafına odaklanmıştır. Bu kapsamda:

- Öğrencinin geçtiği/kaldığı derslere göre **ön koşul uygunluk sorgulama mantığının** geliştirilmesi
- Sorgu sonuçlarının kullanıcıya açık şekilde sunulması (uygun/uygun değil, eksik ön koşullar listesi vb.)
- UI bileşenleri: öğrenci seçme/ekleme, ders durumu görüntüleme, sorgulama ekranları
- Test senaryoları ile doğrulama, hata düzeltmeleri ve kullanıcı deneyimi iyileştirmeleri

1.6. Sistemin Genel Yapısı

BEDES

Başkent E-Dokümantasyon sistemi danışmanların ve öğrencilerin ders seçme, teslim tarihi kontrol gibi süreçleri hızlı ve güvenilir bir şekilde gözlemlmelerine olanak tanır. Sistem derslerin içerik, kategori, kredi, gün saat bilgisi gibi bilgileri kontrol ederek kontenjan durumu ve çakışma kontrolü gibi kritik karar noktalarını gözlemlenebilir sürecin bir parçası haline getirir. Böylelikle dönem başından itibaren benzeri sorunlarla yaşanacak gecikme ve kaynak kayıplarının önüne geçilmiş olur.

Sistemde kullanıcılar temel olarak öğrenci, danışman ve admin rolleri ile beraber işlem yapar. Giriş işlemleri kimlik doğrulama mekanizması yardımı ile gerçekleştirilir. Kullanıcıların erişebileceği ekranlar ve yapabileceği işlemler rol bazlı yetkilendirme ile sınırlıdır. Bu sayede öğrenciler yalnızca kendi ders programı ve bilgilerine erişirken danışmanlar da kendilerine bağlı öğrencileri izleyebilir. Admin ise ders havuzu ve ders oluşturma gibi fonksiyonları kontrol edebilir.

Öğrenci tarafında sistemin ana kullanım noktası “Ders Seçim” ekranıdır. Bu ekranda öğrencilere dersler, gün-saat bilgisi ile birlikte sunulur. Öğrenci bir derse eklemek istediğinde sistem; seçilen dersin şube bilgisi, kontenjan durumu ve mevcut programla zaman çakışması olup olmadığını otomatik olarak kontrol eder. Çakışma bulunması durumunda dersin eklenmesi engellenir. Kontenjan dolu olan derslerde kayıt işlemi engellenir. Eklenen dersler haftalık ders programında gösterilerek görselleştirme sağlanır.

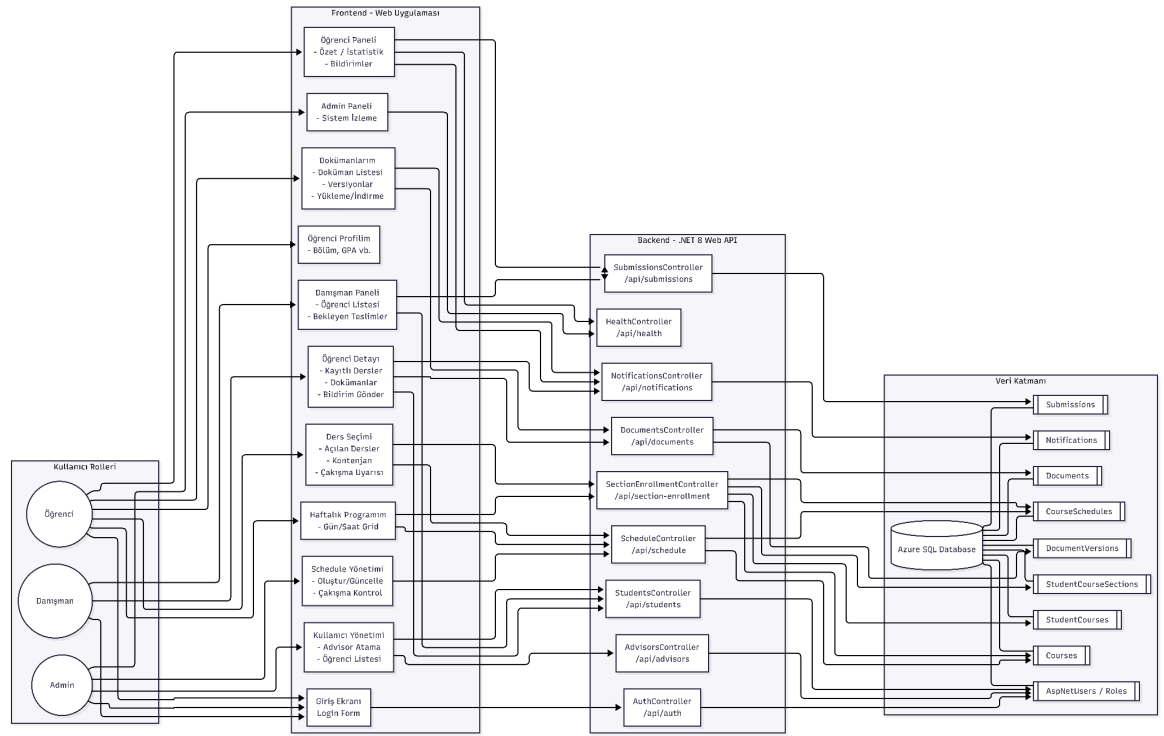
Admin rolü, sistemin veri ve program altyapısını yöneten kritik bileşendir. Dersler ve kategoriler sistem içerisinde tanımlandıktan sonra derslerin hafta içerisindeki dağılımını temsil eden ders çizelgesi verileri oluşturulur ve veritabanında saklanır. Schedule verilerinin doğru olması sistemin ders seçim ekranında yapılacak seçimlerin doğruluğu için temel bir gereksinimdir. Admin ayrıca dersler üzerinde ekleme ve silme yaparak akademik planı yönetebilir.

Danışman rolü, öğrencilerin akademik süreçlerini izleme ve yönlendirme açısından ikinci parçadır. Danışmanlar kendilerine atanmış öğrencilerin kayıt durumlarını, dönem bazında ders programlarını ve temel akademik özetlerini görüntüleyebilir. Sistem danışman ve öğrenci etkileşimlerini desteklemek için bildirim özelliğini kullanabilir.

Sistemin veri akışı genel olarak; derslerin ve schedule'ların veritabanında tutulması, öğrencinin ders seçim ekranında bu bilgileri çekmesi, ders ekleme

talebinde backend'in kontenjan kontrollerini yapması ve sonuçta öğrencinin programına ders eklenmesi şeklinde ilerler. Tüm işlemler standart API uçları üzerinden gerçekleşir; istemci tarafı güncel programı gösterir.

Bu yapı sayesinde Danışmanlık ve Ders Planlama Sistemi, dönem başındaki ders seçimi sürecini daha düzenli hale getirir; öğrencilerin programlarını doğru kurmalarını kolaylaştırır ve danışmanlık süreçlerinde kontrol edilebilir, merkezi bir yönetim altyapısı sunar.



Şekil 1.1. Sistemin Genel Mimarisi

Şekil 1.1.'de görüldüğü üzere, Danışmanlık ve Ders Planlama Sisteminin iş akışı; öğrencilerin ders seçimi ve dönemlik ders programlarını hatasız biçimde oluşturmasını, danışman ve admin tarafında ise süreçlerin merkezi şekilde yönetilmesini amaçlayacak biçimde tasarlanmıştır.

İlk aşamada Admin, web arayüzü üzerinden sistemin akademik veri altyapısını hazırlar. Bu kapsamda dersler ve derslere ait temel bilgiler (ders kodu, adı, kredi/AKTS, kategorisi vb.) sisteme tanımlanır. Ardından derslerin gün-saat, şube (section), derslik/lab ve öğretim elemanı gibi program detaylarını içeren schedule (ders çizelgesi) verileri oluşturulur. Bu veriler, sistemde ders seçimi ve haftalık program ekranlarının doğru çalışabilmesi için kritik öneme sahiptir.

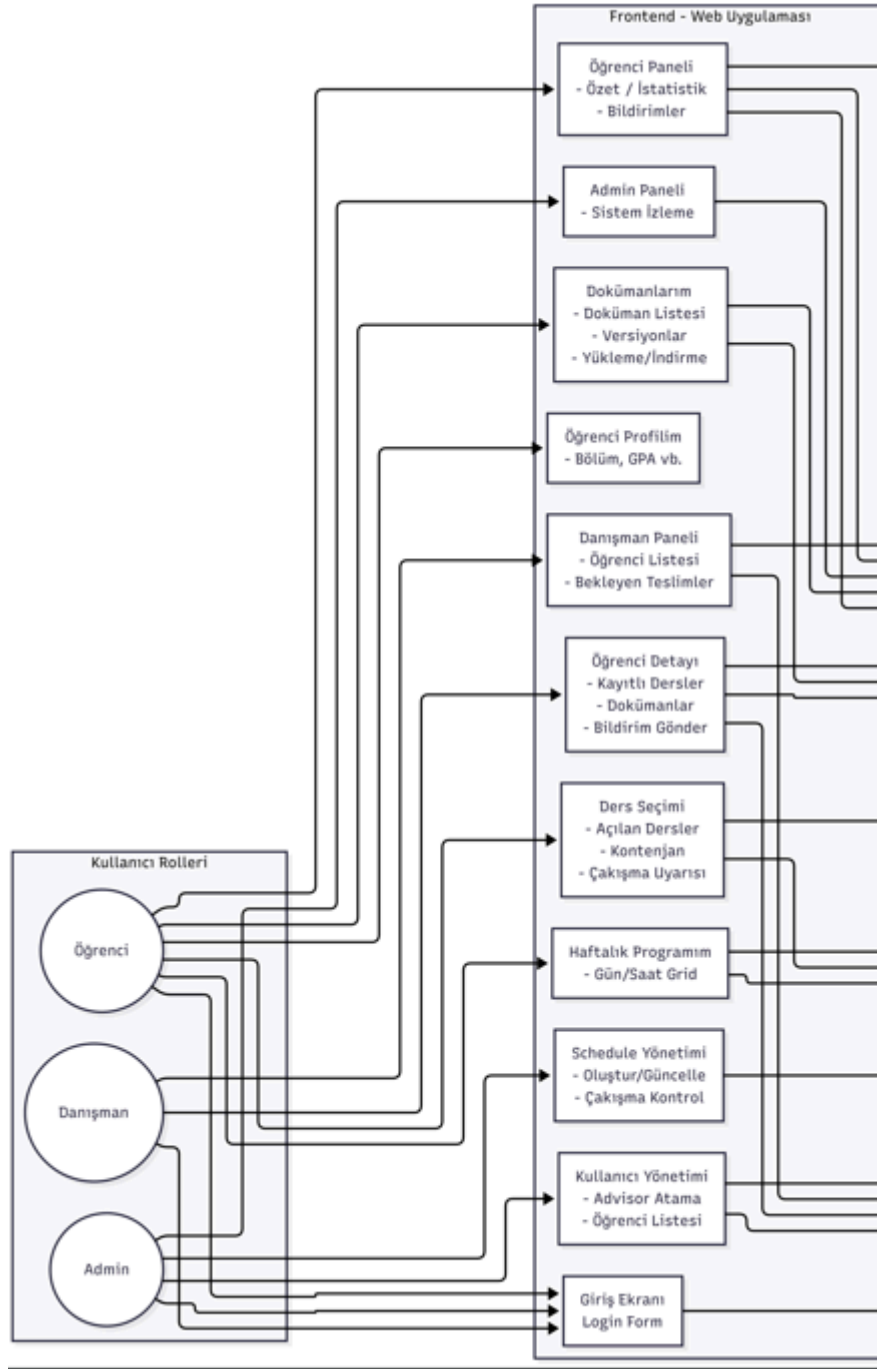
Sonraki aşamada Öğrenciler, giriş ekranı üzerinden kimlik doğrulama yaparak sisteme erişir. Giriş işlemi başarılı olduğunda sistem, kullanıcıya JWT tabanlı bir oturum belirteci sağlar ve bu belirteç ile kullanıcı ekranları güvenli şekilde kullanılabilir hale gelir. Öğrenci, "Ders Seçimi" ekranına girdiğinde sistem; seçilen yarıyla göre dersleri ve derslerin schedule bilgilerini veritabanından çekerek listelemeyi gerçekleştirir. Bu listede derslerin kontenjan doluluk bilgileri ve öğrencinin o ders için kayıt durumu da gösterilir.

Öğrenci bir dersi programına eklemek istediğinde sistem, işlemi gerçekleştirmeden önce iki temel kontrol uygular: zaman çakışması kontrolü ve kontenjan kontrolü. Zaman çakışması kontrolünde, öğrencinin mevcut kayıtlı derslerinin schedule oturumları ile yeni seçilen dersin oturumları karşılaştırılır; aynı gün ve saat aralığında çakışma varsa kayıt reddedilir ve kullanıcıya çakışan ders bilgisi ile birlikte açıklayıcı bir mesaj döndürülür. Kontenjan kontrolünde ise ilgili şubenin dolu olup olmadığı kontrol edilir; kapasite doluysa kayıt işlemi engellenir.

Kontrollerin başarılı olması durumunda öğrenci kaydı veritabanında güncellenir. Kayıt işlemi StudentCourseSections (ders–şube bazlı kayıt) ve gerekli ise StudentCourses gibi öğrenci-ders ilişki tablolarına yansıtılır. Böylece öğrenci programı güncel hale gelir ve “Haftalık Programım” ekranında dersler günlere göre otomatik olarak görüntülenebilir.

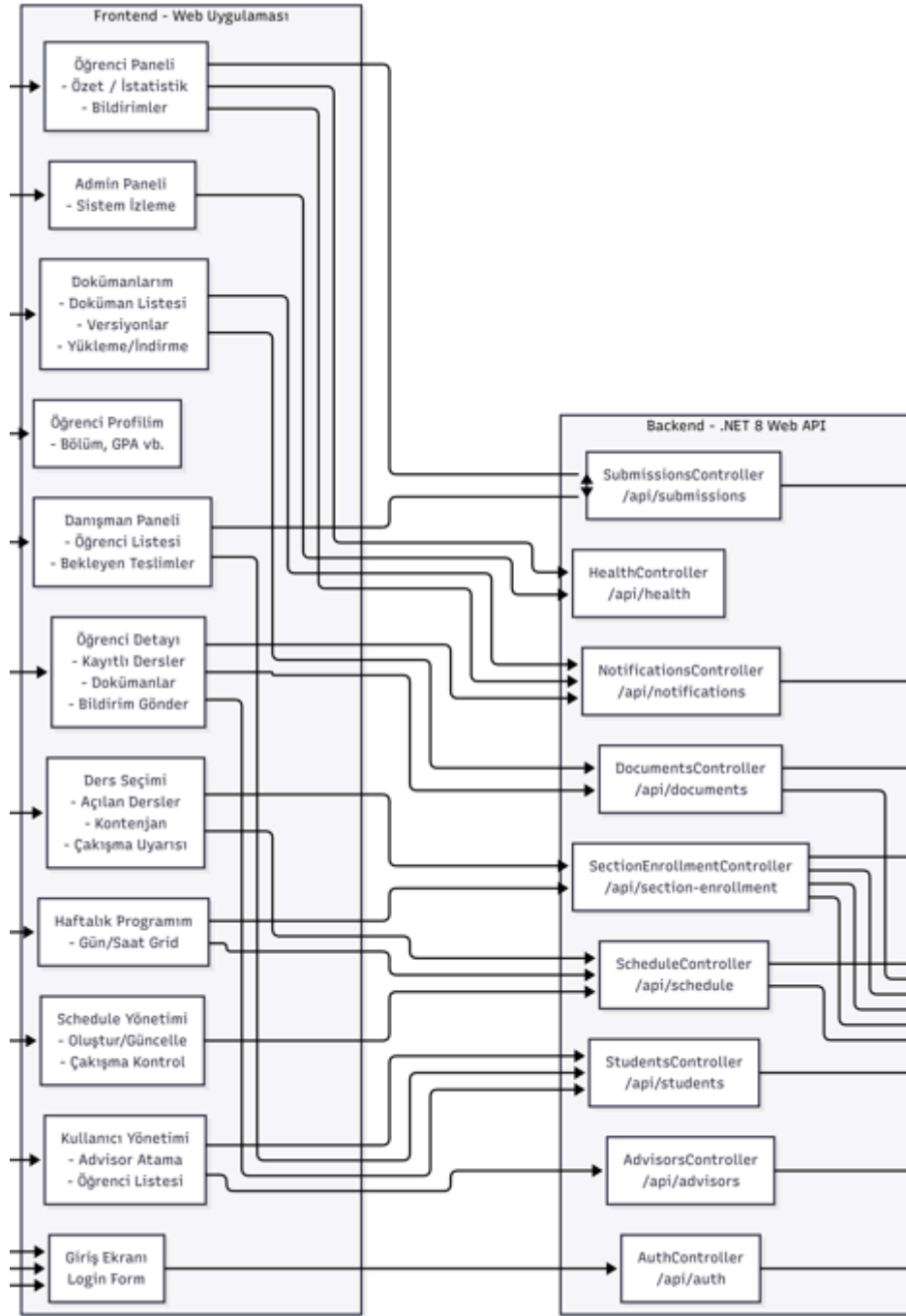
Sistem yalnızca ders ekleme/çıkarma ile sınırlı kalmaz; süreç boyunca danışman ve admin rolleri için de izleme ve yönetim mekanizmaları sunar. Danışmanlar kendi öğrencilerinin kayıt durumlarını takip edebilir; admin ise schedule üretimi, çakışma raporları ve sistem sağlık kontrolleri gibi operasyonları yönetebilir. Ayrıca bildirim mekanizması ile kritik işlemler (atama, hatırlatma, bilgilendirme) kullanıcıya sistem üzerinden iletilebilir.

Bu iş akışı sayesinde sistem; öğrenci tarafında hızlı, kontrollü ve hataya kapalı bir ders seçimi deneyimi sağlarken, yönetim tarafında merkezi, izlenebilir ve sürdürülebilir bir akademik planlama altyapısı sunar.



Şekil 1.2. Kullanıcı Roller ve Sisteme Erişim

Şekilde ilk katman, sistemle etkileşime giren kullanıcıları ve rollerini göstermektedir. Sistem üç temel rol üzerine kuruludur: Öğrenci, Danışman ve Admin. Tüm roller, web uygulaması üzerinden Giriş Ekranı ile kimlik doğrulama sürecini tamamlar. Kimlik doğrulama sonrası kullanıcıya rolüne uygun ekranlar açılır ve yetkisiz erişimler rol bazlı kurallarla engellenir. Bu yapı sayesinde öğrenci yalnızca kendi ders seçimi ve programına erişebilirken; danışman kendi öğrencilerinin süreçlerini takip edebilir, admin ise sistemin yönetsel işlemlerini yürütür.



Şekil 1.3. Frontend Katmanı

Şekil 1.3. ikinci bölümünde web uygulamasındaki ana ekranlar yer alır. Bu ekranlar, kullanıcıların ihtiyacına göre gruplandırılmıştır:

- **Giriş Ekranı:** Kullanıcıların sisteme giriş yaptığı ve oturumun başlatıldığı ekrandır.

Öğrenci Ekranları:

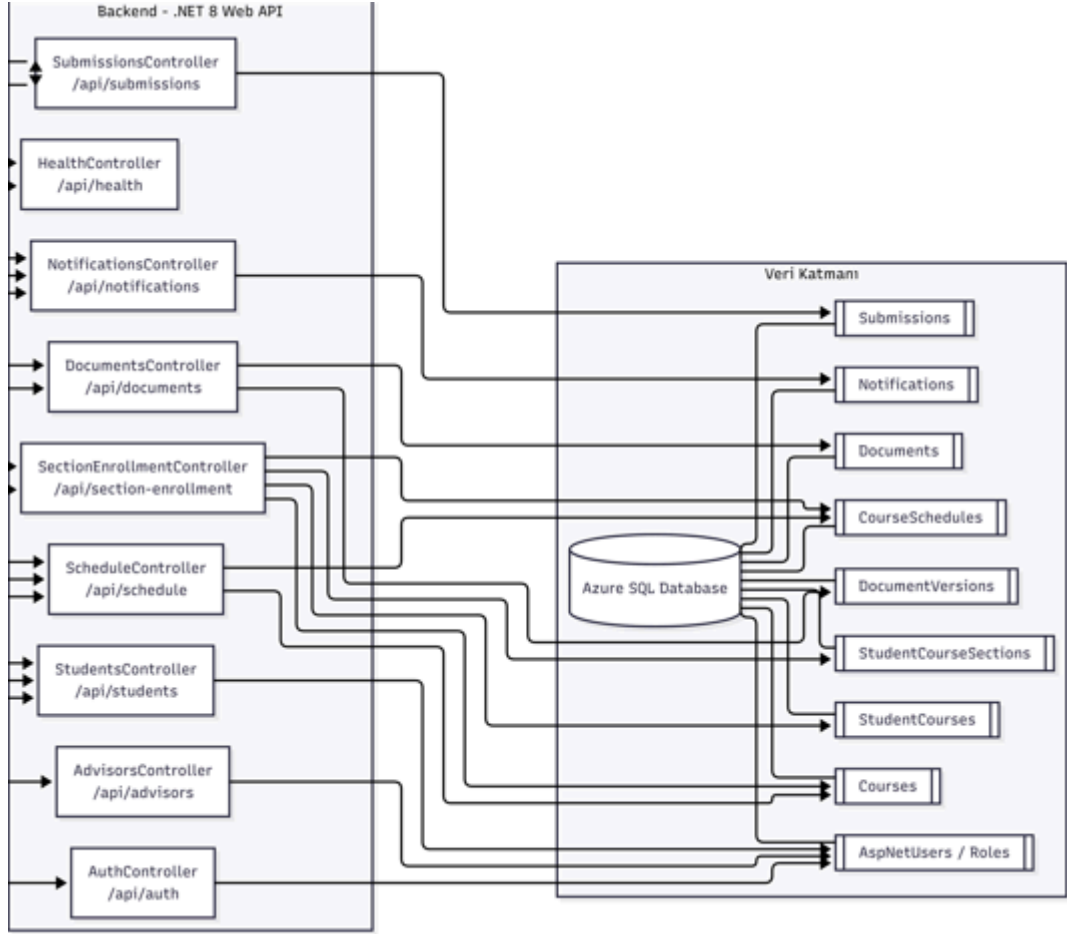
- Öğrenci Paneli: Bildirimler ve genel özet bilgilerin görüntülenmesi.
- Ders Seçimi: Yarıyıl seçimi ile derslerin listelendiği; derslerin gün-saat, şube, kontenjan bilgileriyle gösterildiği ekrandır.
- Haftalık Programım: Öğrencinin seçtiği derslerin günlere göre haftalık tabloda görüntülendiği ekrandır.
- (Sistemde varsa) doküman / profil gibi modüller de öğrencinin kişisel süreçlerini yönetmesini sağlar.

Danışman Ekranları:

- Danışman Paneli: Danışmana atanmış öğrencilerin listesi, özet durumları ve takip edilecek işlerin görünümü.
- Öğrenci Detayı: Seçilen öğrencinin kayıt durumu, programı ve ilgili bilgilerin görüntülenmesi.

Admin Ekranları:

- Admin Paneli: Sistem durumu ve operasyonel özet göstergeler.
 - Schedule Yönetimi: Ders programlarının (schedule) oluşturulması/güncellenmesi ve çakışma kontrollerinin yönetilmesi.
 - Kullanıcı Yönetimi: Danışman atama gibi kritik idari işlemler.
- Bu katman, kullanıcı deneyimini oluşturur; arka planda tüm veriler backend API üzerinden sağlanır.



Şekil 1.4. Backend ve Database Katmanı

Şekil 1.4. de sistemin iş mantığı ve verilerin tutulduğu veritabanı yapısı yer almaktadır. Frontendden gelen istekler burada işlenmekte ve sonuca göre Veritabanından ilgili veri gönderilmektedir. Kimlik doğrulama ve yetkilendirme, Ders seçimi iş kuralları(çakışma kontrolü, kontenjan kontrolü gibi işlemler) burada yönetilir.

Ders ve schedule tanımlama işlemlerinin tamamlanmasının ardından bilgiler, sistemin kullandığı Azure SQL veritabanında ilgili tablolara kaydedilir. Ders içerikleri Courses tablosunda, dersin gün-saat ve şube bazlı oturumları ise CourseSchedules tablosunda saklanır. Bu adım, derslerin düzenli şekilde saklanmasını, güncellenmesini ve dönem bazında hızlı şekilde listelenmesini sağlar.

BMNKU

Electron projesi, öğretmen/akademik danışmanın belirli bir öğrencinin ders geçmişine bakarak hangi dersleri alıp alamayacağını hızlı biçimde değerlendirebilmesi için geliştirilmiş bağımsız bir masaüstü uygulamasıdır (desktop application). Uygulama, öğrencinin geçtiği/kaldığı dersleri temel alır ve hedeflenen dersler için tanımlı ön koşul kurallarını otomatik şekilde kontrol ederek “alabilir / alamaz” sonucunu üretir (eligibility check (*uygunluk kontrolü*)). Böylece danışmanlık sürecinde manuel kontrol ihtiyacını azaltmayı ve karar verme sürecini hızlandırmayı amaçlar.

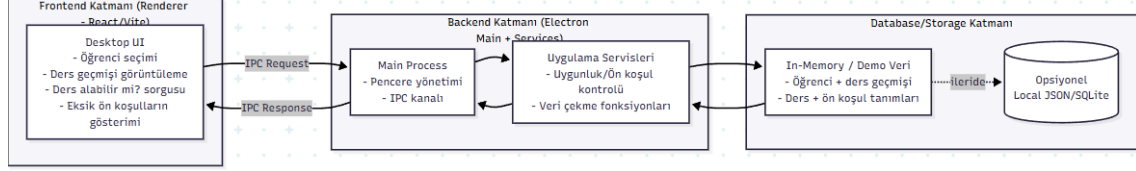
Sistemin genel yapısı Electron’un iki parçalı mimarisi (architecture) üzerine kuruludur. Electron tarafında main process (*ana süreç*) uygulamanın masaüstü çekirdeğini temsil eder; pencere yönetimi (window management), uygulamanın çalıştırılması/kapatılması (lifecycle (*yaşam döngüsü*)) ve gerektiğinde işletim sistemiyle ilgili işlemler bu kısımda yürütülür. Renderer process (*arayüz süreci*) ise kullanıcı arayüzünün çalıştığı katmandır (UI layer (*kullanıcı arayüzü katmanı*)) ve React/Vite ile geliştirilmiştir. Öğretmenin öğrenci seçmesi, öğrencinin ders geçmişini görüntülemesi, kontrol edilmek istenen ders seçmesi ve sonuç ekranlarını görmesi gibi tüm kullanıcı etkileşimleri (user interaction) bu arayüz katmanında gerçekleşir.

Uygulamanın iş mantığı (business logic), ön koşul uygunluk kontrolünü yapan ayrı bir bileşen (component)/katman (layer) olarak kurgulanmıştır. Bu katman; öğrencinin başarılı olduğu dersleri, başarısız olduğu dersleri ve hedef dersin ön koşullarını karşılaştırır (rule evaluation (*kural değerlendirme*)). Sonuç olarak sistem, öğrencinin dersi alıp alamadığını net bir şekilde belirler; eğer alamıyorsa hangi ön koşulların eksik kaldığını da açıklayıcı biçimde listeler (missing prerequisites (*eksik ön koşullar*)). Bu yaklaşım, öğretmenin yalnızca “uygun değil” sonucunu görmek yerine, kararın nedenini de hızlıca anlayabilmesini sağlar.

Veri yönetimi (data management) tarafında uygulama, mevcut aşamada öğrenci bilgilerini ve ders durumlarını hafızada tutacak şekilde tasarlanmıştır (in-memory storage (hafızada saklama)). Demo senaryosunda öğrenci bilgileri uygulama içinde çekilip gösterilebilir ve bu veriler üzerinden uygunluk kontrolü yapılabilir. Böylece uygulama, herhangi bir dış sisteme zorunlu bağımlı olmadan çalışabilen bir karar destek mantığını kanıtlar (proof of concept (kavram kanıtı)). İlerleyen aşamalarda ihtiyaç duyulursa, aynı yapı bozulmadan veri kaynağı genişletilerek dosyadan okuma (file import (dosyadan içe aktarma)) veya bir API üzerinden veri alma (API integration (API entegrasyonu)) gibi yöntemler eklenebilir.

Genel akış açısından bakıldığında sistem şu şekilde işler: Öğretmen uygulamayı açar, değerlendirmek istediği öğrenciyi seçer ve öğrencinin ders geçmişini görüntüler. Ardından kontrol edilmek istenen ders seçilir ve uygunluk sorgusu başlatılır (query (sorgu)). Sistem ön koşulları otomatik kontrol eder ve sonucu arayüzde “alabilir/alamaz” şeklinde sunar; uygun değilse eksik ön koşullar belirtilerek öğretmenin öğrenciyi doğru şekilde yönlendirmesi sağlanır.

Bu yapıyla Electron projesi, danışmanlık sürecinde hızlı karar alınmasını destekleyen, sade ve anlaşılır bir masaüstü uygulama mimarisi sunar.



Şekil 1.4.1. BMNKU Sistem Yapısı

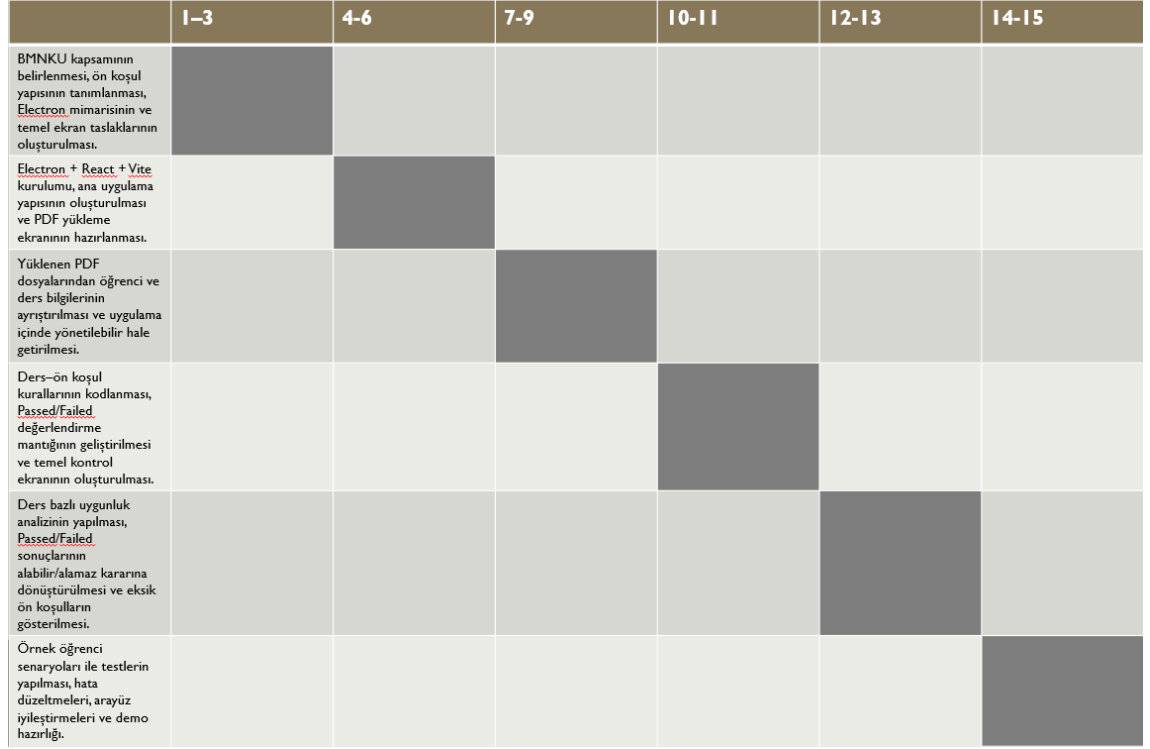
1.7. İş-Zaman Planı

Şekil 1.4.2’de görülen Gantt Çizelgesi 2025-2026 dönemine ait BEDES iş-zaman planlamasını göstermektedir.

	1-3	4-6	7-9	10-11	12-13	14-15
Gereksinim analizi ve sistem tasarımı: roller, veri modeli taslağı, arayüz eskizleri						
Kullanıcı ve danışmanlar için kayıt/giriş modülü: backend API + veritabanı alanları + login ekranı						
Evrak yükleme & görüntüleme çekirdek fonksiyonları: dosya tablosu, upload/download API, upload ekranı						
MNK ve ön koşul kontrol modülünün temel sürümü: kuralların kodlanması, ilgili SQL sorguları, basit kontrol ekranı						
Teslim tarihi tanımlama ve basit bildirim mekanizmasının ilk versiyonu: deadline tabloları, API, bildirim göstergesi						
Veritabanı tasarımının son hali ve temel testler: şema gözden geçirme, örnek senaryoların test edilmesi, hata düzeltme						

Şekil 1.4.2 Gantt Çizelgesi(BEDES)

Şekil 1.4.3’de görülen Gantt Çizelgesi 2025-2026 dönemine ait BMNKG iş-zaman planlamasını göstermektedir.



Şekil 1.4.3 Gantt Çizelgesi(BMNKG)

1.8. Veritabanı Tabloları

AspNetUsers

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	nvarchar(450)	PK	Zorunlu	GUID formatında kullanıcı kimliği
UserName	nvarchar(256)	UQ	Opsiyonel	Kullanıcı adı
Email	nvarchar(256)	UQ	Opsiyonel	E-posta adresi
PasswordHash	nvarchar(MAX)	-	Opsiyonel	Şifrelenmiş parola
AdvisorId	nvarchar(450)	FK → AspNetUsers.Id	Opsiyonel	Öğrencinin danışmanı (self-reference)
CreatedAt	datetime2	-	Zorunlu	Oluşturulma tarihi

Şekil 1.5. AspNetUsers Tablosu

StudentProfiles

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
UserId	nvarchar(450)	FK → AspNetUsers.Id UQ	Zorunlu	1:1 ilişki ile kullanıcı
FirstName	nvarchar(100)	-	Opsiyonel	Ad
LastName	nvarchar(100)	-	Opsiyonel	Soyad
StudentNumber	nvarchar(50)	-	Opsiyonel	Öğrenci numarası
Department	nvarchar(200)	-	Opsiyonel	Bölüm
GPA	float	-	Opsiyonel	Not ortalaması
CompletedCredits	int	-	Opsiyonel	Tamamlanan kredi

Şekil 1.6. StudentProfiles Tablosu

CourseCategories

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
Name	nvarchar(200)	-	Zorunlu	Kategori adı
Description	nvarchar(500)	-	Opsiyonel	Açıklama
DisplayOrder	int	-	Zorunlu	Sıralama

Şekil 1.7. CourseCategories Tablosu

Courses

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
CourseCode	nvarchar(20)	UQ	Zorunlu	Ders kodu (BIL101)
CourseName	nvarchar(200)	-	Zorunlu	Ders adı
TheoryHours	int	-	Zorunlu	Haftalık teori saati
PracticeHours	int	-	Zorunlu	Haftalık uygulama saati
Credits	int	-	Zorunlu	Kredi
ECTS	int	-	Zorunlu	AKTS
CategoryId	int	FK → CourseCategories.Id	Zorunlu	Kategori
Semester	int	-	Opsiyonel	Dönem (1-8)
IsElective	bit	-	Zorunlu	Seçmeli mi?
Description	nvarchar(MAX)	-	Opsiyonel	Ders açıklaması

Şekil 1.8. Courses Tablosu

CourseSchedules

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
CourseId	int	FK → Courses.Id	Zorunlu	Ders
Semester	int	-	Zorunlu	Dönem
SectionCode	nvarchar(10)	-	Zorunlu	Şube (A, B, C)
DayOfWeek	int	-	Zorunlu	Gün (0-6)
StartTime	time	-	Zorunlu	Başlangıç saati
EndTime	time	-	Zorunlu	Bitiş saati
RoomNumber	nvarchar(50)	-	Opsiyonel	Derslik
InstructorName	nvarchar(200)	-	Opsiyonel	Öğretim üyesi
MaxCapacity	int	-	Zorunlu	Kapasite

Şekil 1.9. CourseSchedules Tablosu

StudentCourseSections

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
StudentId	nvarchar(450)	FK → AspNetUsers.Id	Zorunlu	Öğrenci
CourseId	int	FK → Courses.Id	Zorunlu	Ders
SectionCode	nvarchar(10)	-	Zorunlu	Şube
IsCompleted	bit	-	Zorunlu	Tamamlandı mı?
EnrolledAt	datetime2	-	Zorunlu	Kayıt tarihi

Şekil 1.10. StudentCourseSections Tablosu

Documents

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
StudentId	nvarchar(450)	FK → AspNetUsers.Id	Zorunlu	Öğrenci
CourseId	int	FK → Courses.Id	Zorunlu	Ders
SectionCode	nvarchar(10)	-	Zorunlu	Şube
IsCompleted	bit	-	Zorunlu	Tamamlandı mı?
EnrolledAt	datetime2	-	Zorunlu	Kayıt tarihi

Şekil 1.11. Documents Tablosu

DocumentVersions

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
DocumentId	int	FK → Documents.Id	Zorunlu	Ana döküman
VersionNo	int	-	Zorunlu	Versiyon numarası (1,2,3...)
FileName	nvarchar(500)	-	Zorunlu	Dosya adı
ContentType	nvarchar(100)	-	Zorunlu	MIME tipi
Size	bigint	-	Zorunlu	Dosya boyutu (byte)
StoragePath	nvarchar(1000)	-	Zorunlu	Depolama yolu
UploadedByUserId	nvarchar(450)	FK → AspNetUsers.Id	Zorunlu	Yükleyen kullanıcı

Şekil 1.12. DocumentVersions Tablosu

Notifications

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
UserId	nvarchar(450)	FK → AspNetUsers.Id	Zorunlu	Alıcı kullanıcı
Title	nvarchar(200)	-	Zorunlu	Başlık
Message	nvarchar(MAX)	-	Zorunlu	Mesaj içeriği
IsRead	bit	-	Zorunlu	Okundu mu?

Şekil 1.13 Notifications Tablosu

Comments

Kolon Adı	Veri Tipi	Anahtar	Null	Açıklama
Id	int	PK	Zorunlu	Otomatik artan ID
DocumentVersionId	int	FK → DocumentVersions.Id	Zorunlu	Hangi versiyona yorum
AuthorUserId	nvarchar(450)	FK → AspNetUsers.Id	Zorunlu	Yorumu yazan
Content	nvarchar(MAX)	-	Zorunlu	Yorum içeriği

Şekil 1.14. Comments Tablo

İlişki Tablosu

Kaynak Tablo	FK Kolonu	Hedef Tablo	PK Kolonu	İlişki Tipi	Silme Kuralı
AspNetUsers	AdvisorId	AspNetUsers	Id	1:N	RESTRICT
StudentProfiles	UserId	AspNetUsers	Id	1:1	CASCADE
Courses	CategoryId	CourseCategories	Id	1:N	RESTRICT
CourseSchedules	CourseId	Courses	Id	1:N	CASCADE
StudentCourseSections	StudentId	AspNetUsers	Id	1:N	CASCADE
StudentCourseSections	CourseId	Courses	Id	1:N	CASCADE
Documents	OwnerUserId	AspNetUsers	Id	1:N	RESTRICT
Documents	AdvisorUserId	AspNetUsers	Id	1:N	SET NULL
DocumentVersions	DocumentId	Documents	Id	1:N	CASCADE
DocumentVersions	UploadedByUserId	AspNetUsers	Id	1:N	RESTRICT
Comments	DocumentVersionId	DocumentVersions	Id	1:N	CASCADE
Comments	AuthorUserId	AspNetUsers	Id	1:N	RESTRICT
Notifications	UserId	AspNetUsers	Id	1:N	CASCADE
Submissions	StudentId	AspNetUsers	Id	1:N	RESTRICT
Submissions	DocumentId	Documents	Id	1:N	SET NULL
DocumentRatings	DocumentVersionId	DocumentVersions	Id	1:N	CASCADE
DocumentRatings	AdvisorUserId	AspNetUsers	Id	1:N	RESTRICT
ScheduleConflicts	ScheduleId	CourseSchedules	Id	1:N	CASCADE

Şekil 1.15. İlişki Tablosu

2. ARAÇLAR VE YÖNTEMLER

2.1 Arda'nın Görevleri

Arda, projenin web sitesi kısmının hazırlanmasında (backend-frontend-db) görev almıştır. Bu görev kapsamında kullanılan araçlar aşağıdaki gibidir:

.NET8([ASP.NET](#) Core Web API): Backend servis katmanı.

React+Vite: Piyasada sıkça kullanılması, component bazlı kullanıma olan uygunluğu sebebiyle kullanılmıştır. Yapılan değişikliklerin daha hızlı görüntülenebilmesi için Vite kullanılmıştır

React Router: Sayfalar arasında geçişi sağlaması amacıyla kullanılmıştır.

Tailwind: CSS kodlarının daha hızlı ve derli toplu geliştirilebilmesi için kullanılmıştır.

Entity Framework Core 8(Code-first): Veritabanı şeması yönetimi ve ORM.

SQL/Local DB: Geliştirme süresince maliyetten kaçınabilmek amacıyla kullanılan ilişkisel veritabanı sistemidir.

[ASP.NET](#) Core Identity: Kullanıcıların rol yönetimlerinin kontrolünde kullanılmıştır.

Visual Studio Code: Hafif yapısı sebebiyle Frontend'in geliştirilmesinde kullanılmıştır.

JetBrains Rider: Yerleşik git entegrasyonu sebebiyle Backend'in geliştirilmesine kullanılmıştır.

Azure SQL: Projenin tamamlanmış halinde kullanılması hedeflenen ilişkisel veritabanı servsidir.

JWT: Stateless kimlik doğrulama ve oturum yönetimi sağlamıştır.

Swagger: Oluşturulan API uçlarının test edilmesinde ve görüntülenmesinde kullanılmıştır.

Azure Blob Storage: Projenin tamamlanmış sürümünde kullanılması hedeflenen dosya saklama servsidir.

Git: Sürüm kontrol için kullanılmıştır.

Kullanılan yöntemler aşağıdaki gibidir:

Katmanlı Mimari: Frontend-Backend ve veritabanı arasındaki net ayrım sayesinde modüler yapı ve sorunların tespitinin kolaylaşması hedeflenmiştir.

Servis Üzerinden Veri Erişimi: Veri çekme istekleri direkt olarak fetch/axios kullanılarak değil UI değişimini minimize etmek amacıyla servisler aracılığıyla sağlanmıştır.

Rol Bazlı Yetkilendirme: Rol kontrolleri ile yetkiler sınırlandırılmıştır.

Backend Merkezli Kural Kontrol: Çakışma, kontenjan gibi işlemlerin backend tarafında kontrol edilmesi ile frontendde yapılan kritik işlemin azaltılması amaçlanmıştır.

Azure Odaklı Planlama: Projenin tamamlanmış halinde büyük topluluklar tarafından kullanılacağı varsayılarak On-Prem yerine Cloud servisler hedeflenmiştir. Proje içerisinde [ASP.NET](#) kullanılmasının sebeplerinden biri de Microsoft Azure ekosistemi ile uyum içerisinde çalışmasıdır.

2.2 Onur'un Görevleri

Onur, Electron tabanlı masaüstü uygulamasında ön koşul uygunluk kontrolü ve kullanıcı arayüzü (UI) odaklı geliştirme sorumluluğunu üstlenmiştir. Projenin amacı; öğretmenin/danışmanın spesifik bir öğrenciyi seçerek öğrencinin ders geçmişine göre hangi dersleri alıp alamadığını hızlı biçimde görmesini sağlamaktır.

Görev tanımı kapsamında Onur'un sorumlulukları:

- Uygunluk/ön koşul kontrol mantığı: Öğrencinin geçtiği/kaldığı ders bilgilerine göre hedef dersin alınabilirliğini hesaplayan kontrol akışlarının geliştirilmesi.
- Sonuç sunumu: “Alabilir / Alamaz” çıktısının öğretmene anlaşılır şekilde gösterilmesi; “alamaz” durumunda eksik ön koşulların listelenmesi.
- Arayüz bileşenleri: Öğrenci seçimi/görüntüleme, ders uygunluk sorgulama ekranları ve temel etkileşimlerin (butonlar, listeler, uyarılar) geliştirilmesi.
- Demo doğrulama ve hata düzeltmeleri: Demo senaryoları üzerinden kontrol çıktılarının test edilmesi, UI akış hatalarının giderilmesi ve kullanılabilirliğin iyileştirilmesi.

Kapsam (Onur'un etkilediği modüller):

- Öğrenci bilgisi görüntüleme ekranları
- Ders uygunluk sorgulama ekranı
- Ön koşul kontrol çıktıları (uygun/uygun değil + eksik ön koşullar)
- Temel test senaryoları ve demo akışı

Neden Bu Teknolojiler Seçildi? (Electron + React + Vite)

Electron seçimi (Masaüstü hedefi ve dağıtım kolaylığı):

- Uygulama, öğretmen/danışman tarafından masaüstünde çalıştırılacağı için Electron, web teknolojileriyle cross-platform (Windows/macOS/Linux) masaüstü uygulaması üretmeyi mümkün kılar.
- Tek bir kod tabanı ile masaüstü ortamında çalışabilme, demo ve teslim sürecinde kurulum/çalıştırma açısından pratiklik sağlar.

React seçimi (UI geliştirme hızı ve sürdürülebilirlik):

- Öğrenci/ders listeleri, filtreleme, durum kartları gibi arayüzler için React'in bileşen tabanlı yapısı hızlı geliştirme ve tekrar kullanılabilirlik sağlar.
- UI state yönetimi (seçilen öğrenci, seçilen ders, sonuç durumu vb.) React ile okunabilir ve yönetilebilir şekilde kurgulanır.

Vite seçimi (Hızlı geliştirme döngüsü):

- Vite, geliştirme sırasında hızlı başlatma ve hot reload avantajı sağlayarak demo odaklı ilerleyen projelerde üretkenliği artırır.
- Modern build süreci ile React arayüzünü hızlıca derleyip Electron ile entegre etmeyi kolaylaştırır.

2.3 Yaşar'ın Görevleri

Yaşar tarafından geliştirilen Electron tabanlı masaüstü uygulaması, masaüstü ortamda çalışacak şekilde tasarlanmış olup, uygulamanın geliştirilme sürecinde aşağıda belirtilen teknolojilerden yararlanılmıştır:

- Electron.js
Masaüstü uygulamanın temel altyapısı Electron.js kullanılarak oluşturulmuştur. Bu teknoloji sayesinde web tabanlı geliştirme araçları kullanarak Windows, macOS ve Linux işletim sistemlerinde çalışabilen çapraz platform bir uygulama geliştirilmiştir. Uygulama mimarisi kapsamında main ve renderer süreçleri ayrıştırılmış, pencere yönetimi ve uygulama yaşam döngüsü Electron üzerinden kontrol edilmiştir.
- React.js
Uygulamanın kullanıcı arayüzü, bileşen tabanlı bir yapı sunan React.js

kullanılarak geliştirilmiştir. Öğrenci ders durumları, ön koşul kontrolleri ve uygulama ekranları dinamik ve etkileşimli bir yapı içerisinde tasarlanmıştır.

- Vite
React tabanlı arayüzün geliştirme sürecinde hızlı derleme ve modern bir geliştirme ortamı sağlamak amacıyla Vite kullanılmıştır. Vite'in sunduğu Hot Module Replacement (HMR) özelliği sayesinde geliştirme süreci daha verimli hale getirilmiştir.
- JavaScript (ES6 ve sonrası)
Uygulamanın hem Electron ana sürecinde hem de React tabanlı kullanıcı arayüzünde temel programlama dili olarak modern JavaScript kullanılmıştır. Kod yapısı modüler ve sürdürülebilir olacak şekilde tasarlanmıştır.
- Bileşen Tabanlı Uygulama Mimarisi
Uygulama ekranları, kontrol alanları ve yönlendirme yapısı bağımsız ve yeniden kullanılabilir bileşenler şeklinde tasarlanmıştır. Bu mimari yaklaşım, uygulamanın sürdürülebilirliğini ve geliştirilebilirliğini artırmıştır.
- Build, Paketleme ve Hata Ayıklama Araçları
Uygulamanın çalıştırılabilir masaüstü paketleri oluşturulurken Electron Builder ve Vite build süreçlerinden yararlanılmıştır. Geliştirme sürecinde Electron DevTools ve tarayıcı geliştirici araçları kullanılarak hata ayıklama ve test işlemleri gerçekleştirilmiştir.

Neden Bu Teknolojiler Seçilmiştir? (Electron + React + Vite)

Uygulamada kullanılan teknolojiler, masaüstü kullanım hedefi, geliştirme verimliliği ve sürdürülebilirlik kriterleri dikkate alınarak seçilmiştir.

- Electron.js seçimi (Masaüstü hedefi ve dağıtım kolaylığı)
Uygulamanın masaüstü ortamda çalışacak olması nedeniyle Electron.js tercih edilmiştir. Electron, tek bir kod tabanı ile Windows, macOS ve Linux işletim sistemlerinde çalışabilen çapraz platform masaüstü uygulamaları geliştirmeye olanak sağlamaktadır. Bu durum, kurulum ve teslim süreçlerinde kullanım kolaylığı sağlamıştır.
- React.js seçimi (Kullanıcı arayüzü geliştirme ve sürdürülebilirlik)
Öğrenci ve ders listeleri ile ön koşul kontrollerinin yönetilebilmesi amacıyla React.js tercih edilmiştir. React'in bileşen tabanlı yapısı, arayüzün modüler ve okunabilir şekilde geliştirilmesini sağlamıştır. Uygulama içi durum yönetimi React aracılığıyla etkin biçimde

gerçekleştirilmiştir.

- Vite seçimi (Hızlı geliştirme süreci)
Geliştirme sürecinde hızlı başlatma ve anlık geri bildirim sağlamak amacıyla Vite kullanılmıştır. Vite, hızlı derleme ve hot reload desteği ile geliştirme sürecinin verimliliğini artırmıştır.

3.Sonuçlar

3.1.Hedef Özeti

BEDES

Bu projenin temel hedefi, eğitim kurumlarında ders seçimi ve danışmanlık süreçlerinde görülen parçalı yapı ve manuel takibi ortadan kaldırarak akademik planlamayı tek bir web platformu üzerinden daha hızlı, güvenilir ve izlenebilir hale getirmektir. Sistem; öğrencilerin ders seçimi sırasında karşılaştığı program çakışması, kontenjan yönetimi, ön koşul uygunluğu ve mezuniyet koşullarına uygun ders planlama gibi kritik problemleri kurallarla desteklenen otomatik kontrollerle yönetmeyi amaçlar. Böylece hatalı ders kayıtlarının azalması, dönem içi tekrar eden düzeltme işlemlerinin düşmesi ve dönem başı yoğunluğunun daha verimli yönetilmesi hedeflenmektedir.

Proje kapsamında öğrenci, akademik danışman ve yönetici rollerini kapsayan uçtan uca bir yapı tasarlanmıştır. Öğrenciler derslerin gün-saat ve şube bilgilerini net şekilde görüntüleyerek seçim yapabilecek; sistem zaman çakışması analizi ve kontenjan kontrolü ile hatalı kayıtların önüne geçecektir. Akademik danışmanlar, kendilerine atanmış öğrencileri tek panelden izleyerek ders kayıt durumlarını görüntüleyebilecek ve gerektiğinde bildirim/uyarı mekanizması ile süreci destekleyebilmektedir. Yöneticiler ise ders havuzu, ders/kategori/program yönetimi gibi operasyonları merkezi olarak yönetebilecektir.

Modern web mimarisi yaklaşımıyla geliştirilecek sistem; güvenli kimlik doğrulama, rol bazlı yetkilendirme, standart API servisleri ve dijital kayıt/raporlama altyapısı ile sürdürülebilir bir kullanım ve bakım hedefler. Ayrıca proje, süreçleri daha erişilebilir ve hatasız hale getirerek Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçlarından “Nitelikli Eğitim” (SKA-4) hedefiyle uyumlu bir dijital dönüşüm çıktısı üretmeyi amaçlamaktadır.

BMNKU

Bu projenin temel hedefi, öğretmenlerin/akademik danışmanların spesifik bir öğrencinin akademik durumunu hızlı şekilde değerlendirerek öğrencinin hangi dersleri alıp alamayacağını ön koşul kurallarına göre otomatik biçimde görebileceği bağımsız bir masaüstü uygulaması geliştirmektir. Uygulama, öğrencinin geçmiş ders bilgilerini (geçti/kaldı) temel alarak, hedef dersler için gerekli ön koşulların sağlanıp sağlanmadığını analiz eder ve böylece danışmanlık sürecinde yaşanan manuel kontrol, zaman kaybı ve hata riskini azaltmayı amaçlar.

Uygulama kapsamında öğretmen, ilgili öğrenciyi seçerek veya öğrencinin ders geçmişini sisteme girerek öğrencinin mevcut akademik durumunu temsil edebilecektir. Ardından sistem, derslerin ön koşul yapılarını dikkate alarak öğrencinin her bir ders için “alabilir / alamaz” sonucunu üretecek; alamadığı derslerde ise eksik ön koşulları açık ve anlaşılır biçimde listeleyerek karar verme sürecini kolaylaştıracaktır. Bu sayede ders planlama ve yönlendirme süreçleri daha tutarlı, hızlı ve izlenebilir bir hale getirilecektir.

Teknik olarak uygulama Electron + React/Vite ile geliştirilecek; kullanıcı dostu bir arayüz üzerinden öğrenci seçimi, ders geçmişi görüntüleme ve uygunluk sorgulama işlevlerini sunacaktır. Proje sonunda hedeflenen çıktı, danışmanların ders uygunluğu değerlendirmesini pratikleştiren ve öğrencinin alabileceği dersleri net biçimde gösteren bir masaüstü karar destek uygulamasıdır.

3.2 Hedeflerin Gerçekleştirilmesi

1) Web Projesi – BEDES (ASP.NET + React/Vite + Azure)

Web projesinde hedeflenen mimariye uygun şekilde geliştirme süreci başlatılmış ve sistemin temel iskeleti oluşturulmuştur. Bu kapsamda:

- Proje yapısı kurulmuş, frontend (React/Vite) ve backend (ASP.NET) tarafında temel geliştirme ortamı hazırlanmıştır.
- Rol bazlı yapı ve ekran akışlarına yönelik ilk taslaklar/altyapı çalışmaları yapılmıştır.
- Ders programı, ders seçimi ve yönetim süreçlerine ait modüllerin geliştirilmesine başlanmıştır; sistem local ortamda çalışır hale getirilmiştir.

Ancak projenin planlanan hedeflerinden biri olan Azure veritabanı entegrasyonu henüz gerçekleştirilmemiştir. Bu nedenle verilerin kalıcı biçimde tutulması ve sistemin bulut altyapısı üzerinden tam fonksiyonel çalışması bu aşamada tamamlanmamıştır.

2) Electron Projesi – BMNKU (Electron + React/Vite)

Electron projesinde temel hedeflere yönelik çalışır durumda bir demo geliştirilmiştir. Mevcut durumda:

- Uygulama, belirli bir öğrenciye ait bilgileri sistemden çekebilmekte/görüntüleyebilmektedir.
- Öğrencinin akademik durumuna göre ders uygunluk (alabilir/alamaz) kontrolü yapılabilmektedir.
- Ön koşul/durum kontrolünün iş mantığı temel seviyede çalışır hale getirilmiştir ve demo üzerinden doğrulanabilmektedir.

Bu aşamada uygulama, hedeflenen karar destek yaklaşımının çekirdeğini göstermekte; kapsamın genişletilmesi (daha fazla ders/ön koşul senaryosu, arayüzün tamamlanması, kapsamlı testler vb.) bir sonraki geliştirme adımlarına bırakılmıştır.

3.3 Testler

Sistem üzerinden eklemeli bir çalışma modeli benimsendiğinden her yeni eklenen özellikle beraber önceki özelliklerde sorun çıkma ihtimali doğmuştur. Bununla alakalı olarak geriye dönük şekilde smoke testleri yapılmıştır. Testlerin büyük kısmında sorunlar ortaya çıkmış ve bunla bağlantılı olarak önceki adımlar gözden geçirilmiştir. Özellikle ders ve ders programı tablolarında sorunlar yaşanmıştır. Bunlara yönelik olarak veritabanındaki bazı alanlar değiştirilmiş ve farklı fonksiyonlar yazılmıştır. Projenin eldeki en son halinde yapılan çeşitli 29 testten 1 tanesi başarısızlık sonucu alındıktan sonra düzeltilmiştir. Akla gelen senaryolarda gerek frontend üzerinden arayüz kullanılarak gerekse swagger üzerinden manuel olarak testler yapılmıştır. Bilinen ve aklımıza gelmekte olan testlerin hiçbirinde bir hata alınmamaktadır. Manuel testler haricinde script ile yapılan testler:

-Mevcut derse tekrar kayıt olmayı deneme, dersten çıkış, derse kayıt, ders bulunamama durumu, kayıtlı dersleri getirme

-Ders saatlerinde çakışma kontrolü, farklı günler arasındaki saatlerin bağımsızlığının kontrolü, farklı şubeler arası çakışma kontrolü, çakışma olduğu durumda sistemin yanıtı

-Kontenjan dolu olan derse kayıt, kontenjan kontrolü, kontenjanın azalma kontrolü

-Ders programı oluřturma, akıřma kaydı

konularını ierir řekilde testler bulunmaktadıř. Gzden kaırılan hatalar ve olası uyum sorunları yeni zellikler eklendike test edilerek giderilecektir. Yapılan testler sonrasında bir hata grnmemekle beraber sistem demoya hazır hale getirilmiřtir.

4. ARAYÜZ FOTOĞRAFLARI VE ÖZELLİKLERİ

Giriş Ekranı

BEDES

Hoşgeldiniz

Hesabınıza giriş yapınız

E posta

you@example.com

Şifre

Giriş Yap

Hesabın yok mu? [Kayıt Ol](#)

Demo Hesaplar:

Öğrenci: studentnumarası@local / Student123!

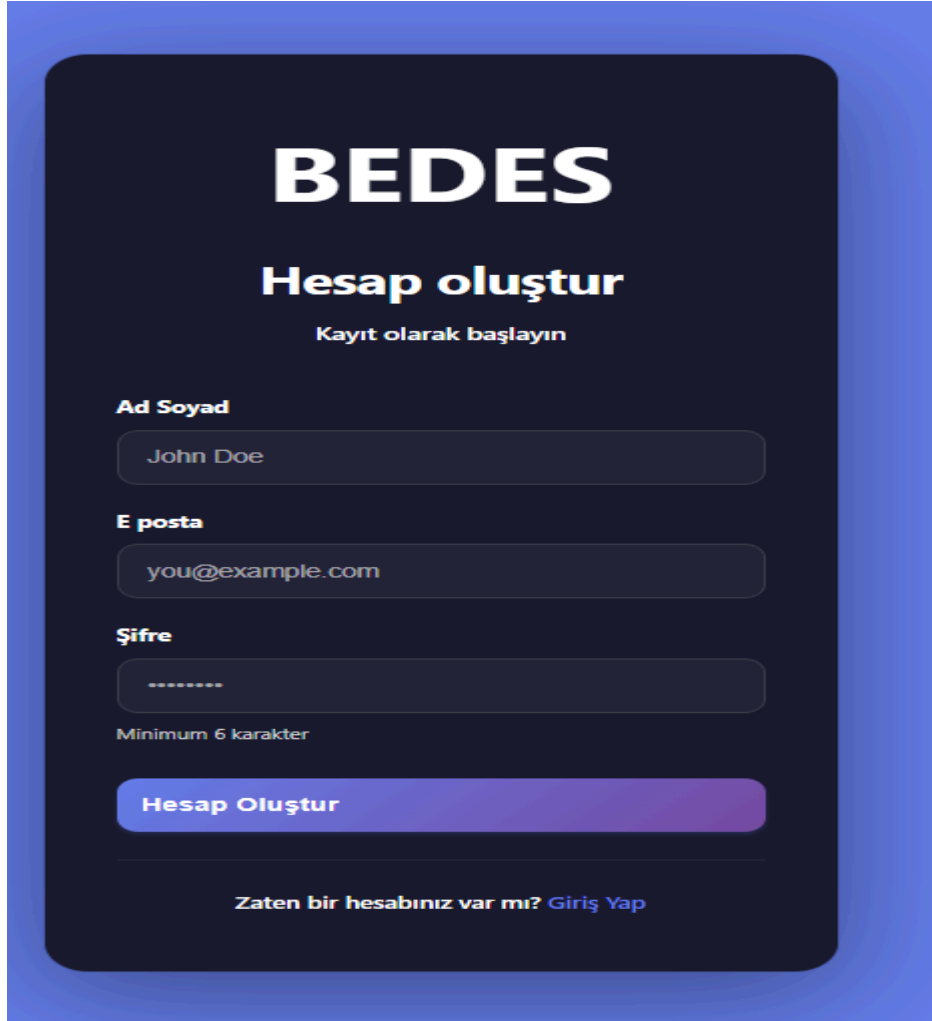
admin: admin@local / Admin123!

Danışman: advisornumarası@local / Advisor123!

Şekil 2.1. Giriş Ekranı

Sistemin güvenli giriş noktasını oluşturur. JWT tabanlı token authentication mekanizması kullanılmaktadır. Kullanıcı e-posta ve şifre bilgilerini girdikten sonra, backend API'den alınan access token localStorage'a kaydedilir ve sonraki tüm API isteklerinde header da kullanılır. Bu sayede stateless ve güvenli bir kimlik doğrulama sistemi kullanılmıştır. Sistem üç farklı kullanıcı rolünü desteklemektedir. Form validasyonu ile boş alan kontrolü yapılmaktadır. Giriş sonrasında kullanıcılar React Router DOM ile role-based routing mantığına göre kendilerine ait dashboard ekranına yönlendirilir.

Kayıt Ekranı

The image shows a registration screen for a system named 'BEDES'. The screen has a dark blue background with a lighter blue border. At the top, the word 'BEDES' is written in large, bold, white capital letters. Below it, the text 'Hesap oluştur' (Create Account) is written in bold white, followed by 'Kayıt olarak başlayın' (Start by registering) in a smaller white font. There are three input fields: 'Ad Soyad' (Full Name) with the value 'John Doe', 'E posta' (Email) with the value 'you@example.com', and 'Şifre' (Password) with a masked value '*****'. Below the password field, there is a note 'Minimum 6 karakter' (Minimum 6 characters). A large, rounded rectangular button with a blue-to-purple gradient and the text 'Hesap Oluştur' (Create Account) is positioned below the input fields. At the bottom, there is a link that says 'Zaten bir hesabınız var mı? Giriş Yap' (Do you already have an account? Log In).

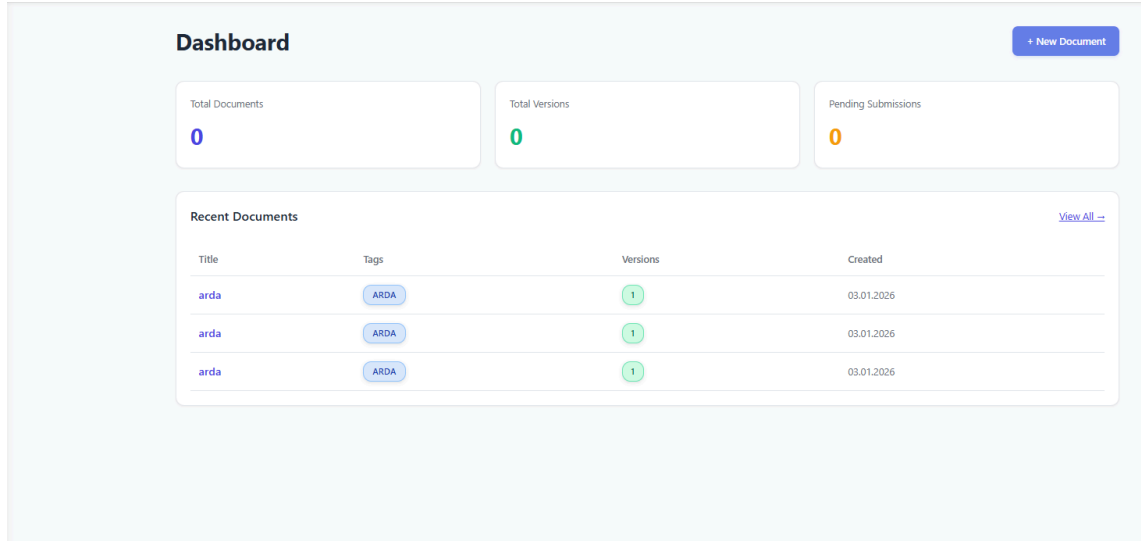
Şekil 2.2. Kayıt Ekranı

Yeni kullanıcıların sisteme kaydolmasını sağlayan register ekranı, login ekranı ile benzer bir tasarıma sahiptir. Ad-Soyad, E-posta ve şifre olmak üzere üç temel bilgi ile kayıt işlemi gerçekleştirilir.

Frontend tarafında React useState hook'ları ile form state yönetimi yapılmıştır. Şifre alanı için minimum 6 karakter kuralı uygulanmakta ve bu kural HTML5 içerisindeki minLength kullanılarak zorunlu kılınmaktadır. E-posta alanı için type="email" özelliği kullanılarak browser seviyesinde validasyon sağlanmıştır.

Asenkron işlem yönetimi ile kayıt işlemi sırasında state loading olarak kaydedilir ve buton devre dışı bırakılır. Bu sayede duplicate kayıtlar önlenir. Backend tarafından başarılı kayıt mesajı geldikten bir süre sonra kullanıcı otomatik olarak login sayfasına yönlendirilir. Kayıt sırasında bir hata olması durumunda backend tarafından gelen hata kodları anlaşılır bir dil ve görüntü ile kullanıcıya bildirilir.

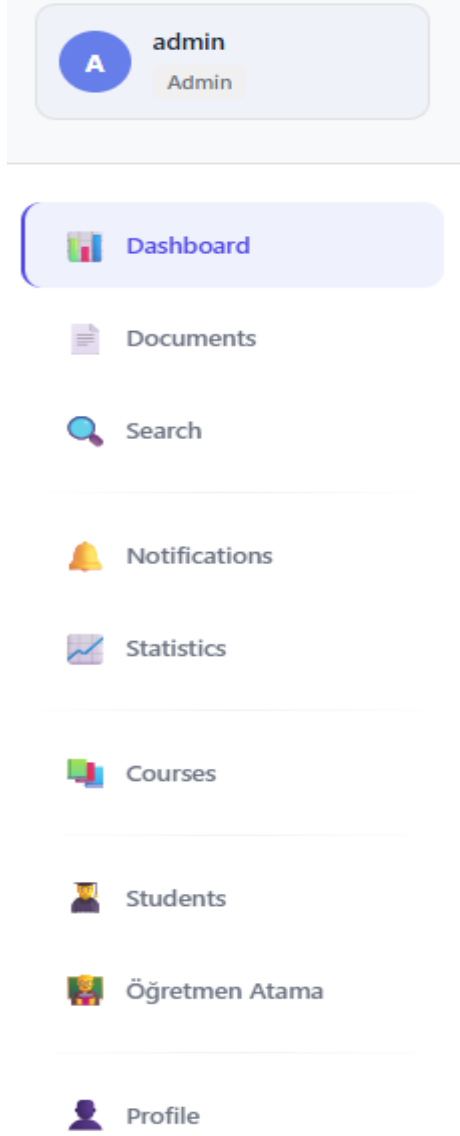
Dashboard Ekranı



Şekil 2.3. Dashboard Ekranı

Bu ekran, kullanıcılar sisteme giriş yaptıktan sonra karşılaştıkları ana yönetim ve gösterim panelidir. Bu ekranda yüklenen dokümanlar ile ilgili temel istatistikler görüntülenebilir. Ekran, giriş yapan kullanıcının rolüne göre farklı içerikler göstermektedir. Örnek olarak öğrenciler kendi yüklediği dosyaları görebilirken, admin yüklenen tüm dosyaları görüntüleyebilir. useState hookları kullanarak istatistikler, geçmiş sürümler gibi veriler yönetilir. Try-catch blokları ile hata yönetimi yapılır. Herhangi bir API çağrısının başarısızlığı durumunda sadece ilgili bölüm gösterilmez.

Navbar Ekranı



Şekil 2.4. Navbar

Site içerisinde bulunan farklı sekmelerde gezinmek için kullanılan ufak bir ekrandır. Giriş yapmış olan kullanıcının rolü kontrol ederek sadece erişebilmesi istenen sayfalara ulaşım sağlar.

Belge Görüntüleme Ekranı

 Belgeler

Tüm Belgeler

Filtrele

Başlık

Başlık ara...

Başlangıç Tarihi

gg . aa . yyyy

Bitiş Tarihi

gg . aa . yyyy

arda



1 versiyon

03.01.2026

 Detayları Gör

arda



1 versiyon

03.01.2026

 Detayları Gör

arda



1 versiyon

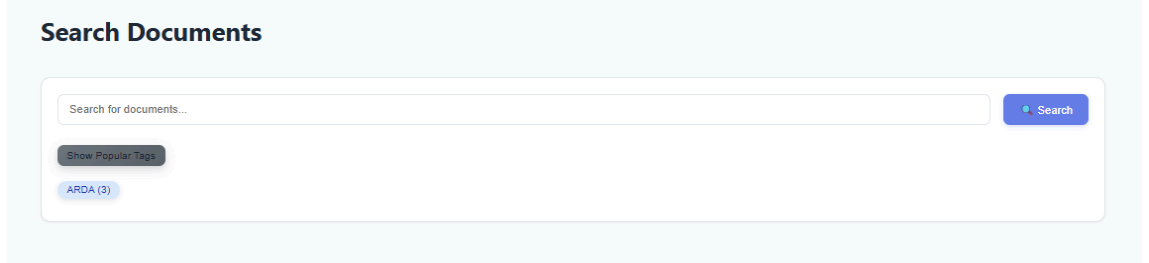
03.01.2026

 Detayları Gör

Şekil 2.5. Belge Görüntüleme

Belge görüntüleme ekranında belgeler tarihlerine ve isimlerine göre filtrelenecek görüntülenebilmektedir. Böylelikle ismi bilinmeyen ancak bulunması gereken belgelere ulaşılabilir. Bulunan belgeler üzerinde detayları gör butonuna tıklanarak ilgili belgeyle ilgili detay ekranına yönlendirilir.

Belge Arama Ekranı



Search Documents

Search for documents...

Search

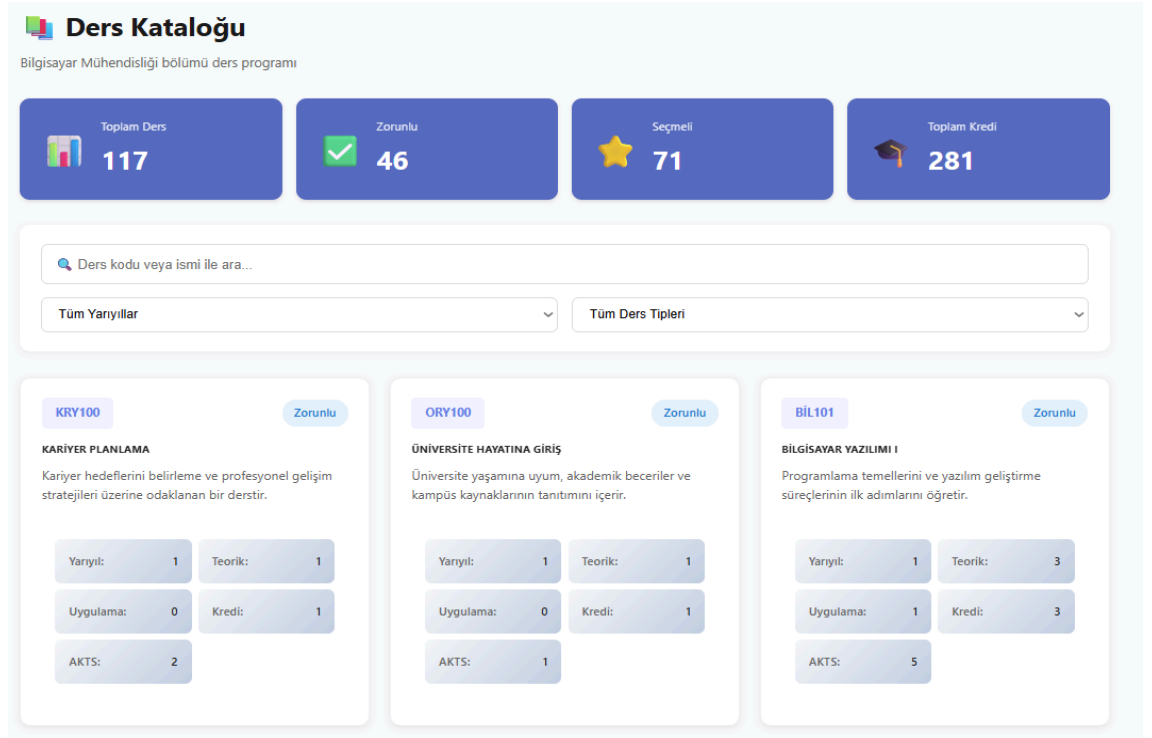
Show Popular Tags

ARDA (3)

Şekil 2.6. Belge Arama

Bu ekran üzerinde kullanıcılar belgelerin başlığına göre arama yapıp o belgeye gidebilir. Arama işlemi partial search desteklemektedir.

Ders Kataloğu Ekranı



Şekil 2.7. Ders Kataloğu

Bu sayfa içerisinde, bölümümüz dahilinde bulunan tüm dersler verileri ile beraber projenin açılışında seedlenmiştir. Gerçek zamanlı olarak Backend'den ders verileri çekilir ve normalize edilerek bu ekranda görüntülenir. Backend'den gelen ders verileri farklı isimlendirmelere sahip olabileceğinden (ve geliştirme boyunca çok değişiklik yapıldığından) Map fonksiyonu ile bu farklılıklar normalize edilir ve frontend ile veri tutarsızlığı oluşmasının önüne geçilir.

Bu ekranda üç farklı filtreleme kriterine göre arama yapılmaktadır. Aramalar partial matching desteklemektedir. Derslerin tercih türüne göre veya hangi dönem içerisinde alacaklarına göre filtreleme yapılabilir. Toplam kredi, seçmeli ve zorunlu ders sayısı gibi basit istatistikler burada görüntülenebilir.

Öğrenci Yönetim ve Görüntüleme Ekranı

The screenshot displays the 'Students Management' interface. At the top, there's a purple header with the title 'Students Management' and a subtitle 'Manage students, advisors, and notifications'. A 'Send Notification' button is located in the top right. Below the header, there are three tabs: 'All Students' (selected), 'Without Advisor', and 'Pending Submissions'. The main area shows three student profiles, each with a checkbox, email, status (ACTIVE), and a 'Send Notification' button. The profiles are for student1@local, student2@local, and student3@local. Each profile includes fields for Registration No, Department, Joined date, and Advisor. Student1 and Student3 have 'Not Assigned' advisors, while Student2 has 'advisor2@local'.

Student	Status	Registration No	Department	Joined	Advisor
student1@local	ACTIVE	N/A	N/A	Invalid Date	Not Assigned
student2@local	ACTIVE	N/A	N/A	Invalid Date	advisor2@local
student3@local	ACTIVE	N/A	N/A	Invalid Date	Not Assigned

Şekil 2.8. Öğrenci Yönetim ve Görüntüleme

Bu ekran içerisinde adminler tüm öğrencileri görüntüleyebilir, onları advisor sahipliği durumuna göre filtreleyebilir ve seçtikleri öğrencilere bildirim yollayabilirler. Öğrencilerin bilgileri bölümünde henüz tüm alanlar hazır olmadığından belirsiz olarak işaretlenmiştir.

Danışmanlar bu ekranda sadece danışmanı oldukları öğrencileri görüntüleyebilir. İşlev olarak adminlerin gördüğü ekrandan bir farklılık bulunmamaktadır. Sadece filtreleme işleminde danışmanı olmayan öğrenciler diye bir sekme görüntüleyememektedirler.

Danışman Atama Ekranı

Öğretmen Atama Yönetimi

Tüm öğrencileri görüntüleyin ve öğretmen ataması yapın

Toplam Öğrenci

3

Öğretmeni Olan

1

Öğretmeni Olmayan

2

Toplam Öğretmen

3

Tümü Öğretmensizler Yenile

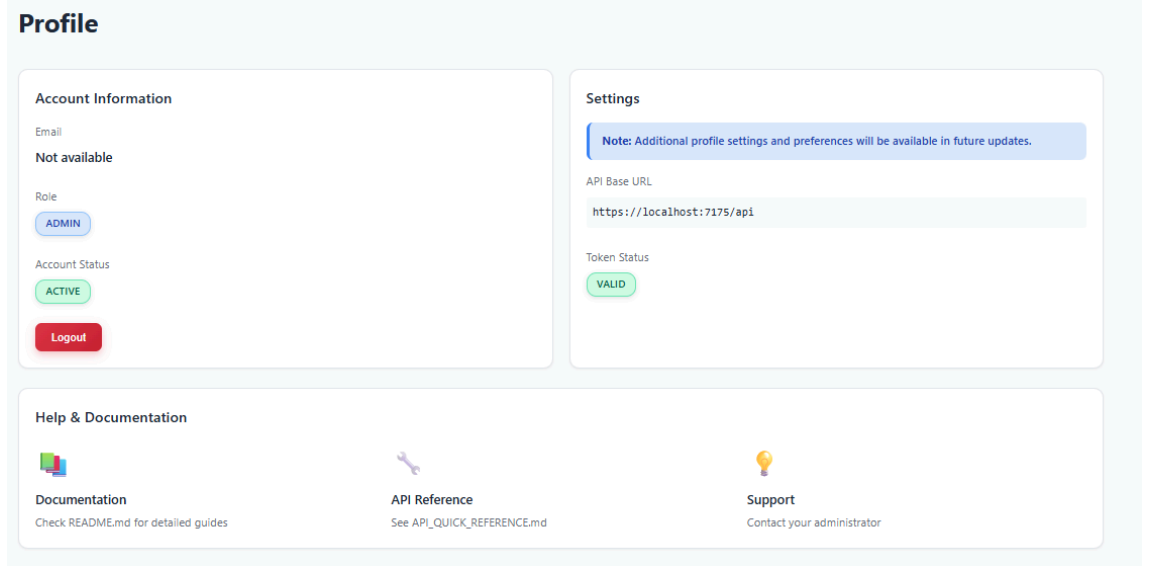
3 öğrenci gösteriliyor

Öğrenci	Email	Belge Sayısı	Durum	Öğretmen	İşlemler
student1@local	student1@local	0	Atanmadı	Atanmamış	Ata
student2@local	student2@local	1	Atandı	advisor2@local	Değiştir Kaldır
student3@local	student3@local	0	Atanmadı	Atanmamış	Ata

Şekil 2.9. Danışman Atama

Sadece adminler tarafından görüntülenebilen bu ekranda danışmanların öğrencilere ataması yapılmaktadır. Bu ekranda öğrencilerin e posta bilgileri, kaç belge yükledikleri, danışmanı olup olmadığı ve var ise kim olduğu görüntülenmektedir. Admin bu ekranı kullanarak ilgili butona tıklayarak istediği öğrenciye müsait danışmanlardan birini atayabilir veya bu danışmanın öğrenci ile bağlantısını kesebilir. Ekranın üst tarafında danışmanı olan ve olmayan öğrenci sayısı ile danışman sayısı özet biçimde yazmaktadır.

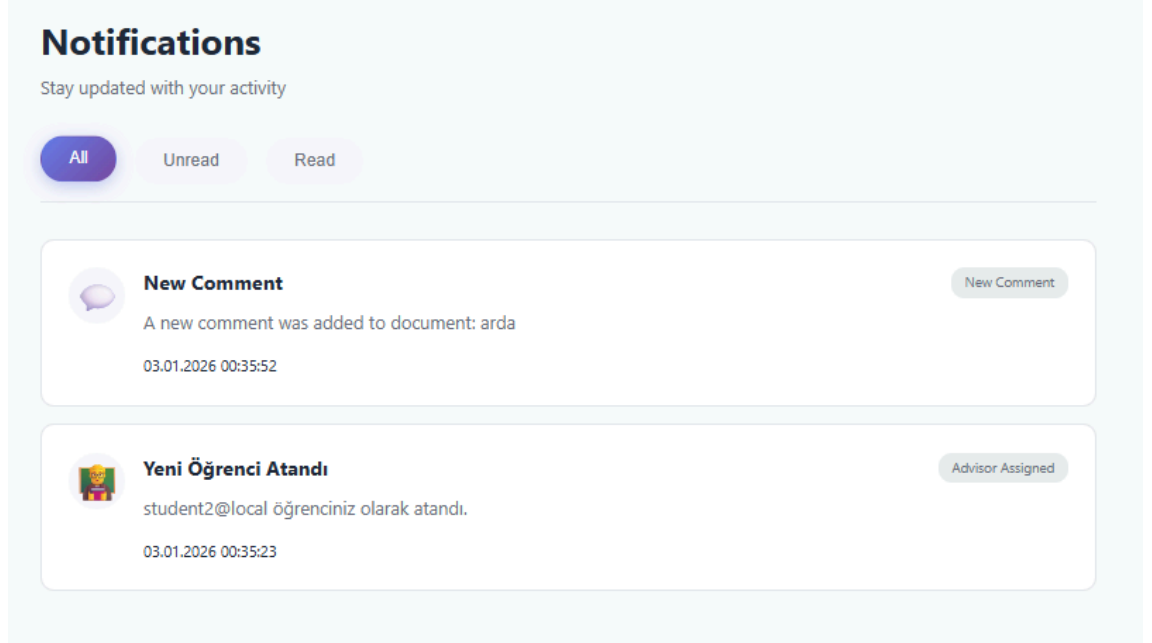
Profil Ekranı



Şekil 2.10. Profil

Profil ekranı, iki ana sütuna ayrılmıştır. Sol tarafta genel hesap bilgileri verilirken sağ tarafta ise öğrencilere yönelik olarak danışman bilgileri, ve iletişim düğmesi bulunacaktır. Kullanıcı verileri localStorage üzerinden okunarak gösterilir. Kullanıcının JWT tokeni kullanılarak decode edilmiş bilgiler çekilir. Logout butonuna tıklandığında localStorage temizlenir ve login sayfasına yönlendirme yapılır.

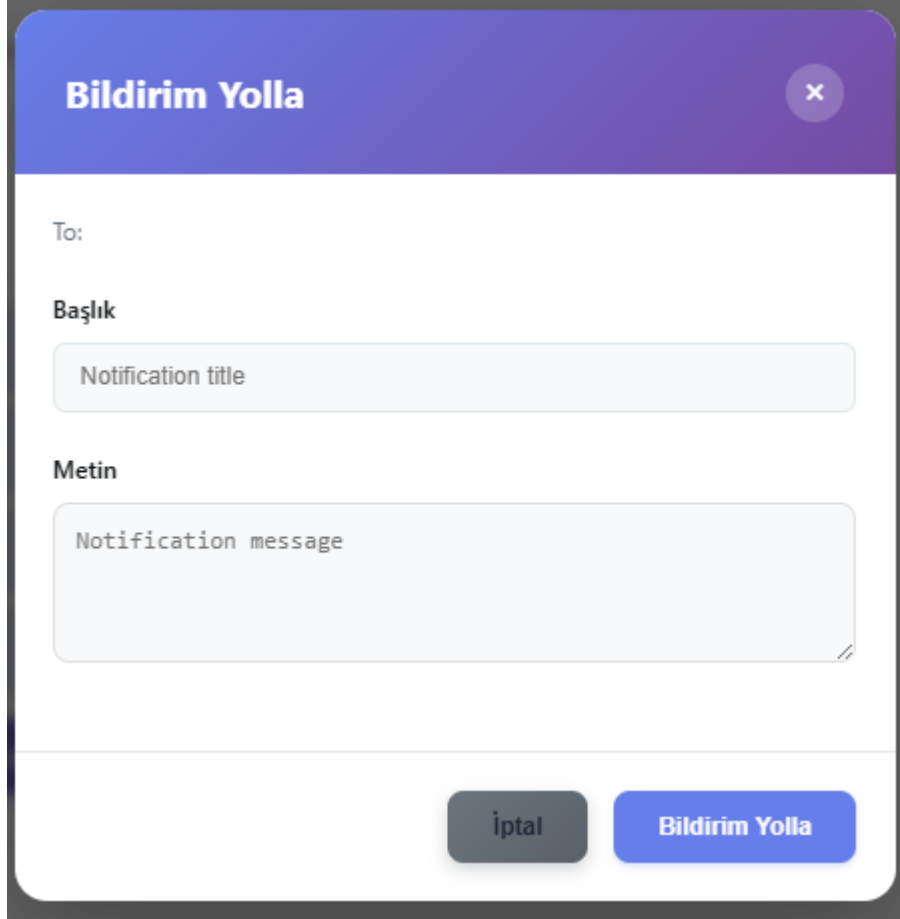
Bildirim Ekranı



Şekil 2.11. Bildirim

Kullanıcılar bu ekranda kendilerine yollanmış olan bildirimleri görebilirler. Öğrenciler, danışmanların ve adminlerin yolladığı bildirimleri, kendilerine verdikleri son teslim tarihine sahip ödev ve benzeri projeleri, yaptıkları yorumların bildirimlerini vb. şeyleri görebilir. Danışmanlar öğrencilerin yaptığı yorumları ve belgeler ile olan diğer etkileşimlerini görebilir. Danışmanlar bu sayfadan hangi öğrenciye atandıklarını görebilir. Öğrenciler ise hangi danışmanın kendilerine atandığını bu ekrandan görebilir.

Bildirim Yollama Ekranı



Bildirim Yolla

To:

Başlık

Notification title

Metin

Notification message

İptal Bildirim Yolla

Şekil 2.12. Bildirim Yollama

Danışmanlar veya adminler bir öğrenciye bildirim yollamak istediği zaman bu ekrana yönlendirilir. Ekranda şu anda bildirim metni ve bildirim başlığı belirtilmektedir. Bildirim yollama butonu ile söz konusu öğrenciye ilgili metin ve başlığı içerir şekilde bildirim yollanır.

Teslim Ekranı

 **Teslim Talebi Oluştur**

Öğrenciye tarih belirterek belge teslimi talep edin

Öğrenci E-postası *

ornek@ogrenci.edu.tr

Öğrencinin e-posta adresini girin

Belge (İsteğe Bağlı)

Belge seçilmedi

İsteğe bağlı: Belirli bir belge için talep oluşturabilirsiniz

Teslim Tarihi *

gg - aa - yyyy

Teslim tarihinden 3 gün önce otomatik hatırlatma gönderilir

Notlar

Ek açıklamalar veya talimatlar yazın...

Öğrenciye gönderilecek ek bilgiler

Teslim Talebi Oluştur

 **Bilgilendirme**

- Öğrenciye anında bildirim gönderilir
- Teslim tarihinden 3 gün önce otomatik hatırlatma yapılır
- Öğrenci, teslim durumunu sisteme işaretleyebilir
- Danışmanlar sadece kendi öğrencileri için talep oluşturabilir

Şekil 2.13. Teslim

Danışmanlar bu ekranı kullanarak öğrencilerinden teslim talebinde bulunabilir. Teslim talebi içerisinde teslim edilen dosya, teslim edecek öğrencinin e postası, son teslim tarihi ve teslim için istenebilecek ek notları barındıran bölgeler bulunur. Her öğrenci için teslimler ayrı olarak belirlenir. Teslim tarihine 3 gün kaldığında son uyarı amaçlı olarak öğrenciye otomatik olarak tekrardan bildirim yollanacaktır.

Teslim Takibi Ekranı

My Submissions

Submission #1

PENDING

Due Date

24.01.2026

Mark as Completed

Şekil 2.14. Teslim Takibi

Öğrenciler kendilerine atanan teslimleri buradan gözlemleyebilir. Gerekli koşulları yerine getirdikleri takdirde tamamlandı olarak işaretlenebilir. Gelecek sürümlerde tamamlanma koşulları danışman tarafından kontrol edilebilir şekle getirilecektir.

Ders Ekleme Ekranı

Ders Seç - Yarıyıl 1

Ders kodu veya ismi ile ara...

BİL101 - BİLGİSAYAR YAZILIMI I

Zorunlu

Kayıt Ol

Şube: A | 3 Kredi | 5 ECTS | T:3 U:1

Programlama temellerini ve yazılım geliştirme süreçlerinin ilk adımlarını öğretir.

Prof. Dr. Ahmet Yılmaz

Ders Saatleri:

- Pazartesi: 08:00 - 09:00 | A100 | Teori
- Pazartesi: 09:00 - 10:00 | A101 | Teori
- Pazartesi: 10:00 - 11:00 | A102 | Teori
- Pazartesi: 11:00 - 12:00 | LAB4 | Uygulama

2/50 - 48 Boş Yer

BİL105 - PROGRAMLAMA LABORATUVARI I

Zorunlu

✓ Kayıtlı

Dersten Çık

Şube: A | 1 Kredi | 2 ECTS | T:0 U:2

Temel programlama kavramlarının uygulamalı olarak pekiştirildiği laboratuvar dersidir.

Öğr. Gör. Can Şahin

Ders Saatleri:

- Pazartesi: 13:00 - 14:00 | LAB5 | Uygulama
- Pazartesi: 14:00 - 15:00 | LAB1 | Uygulama

4/30 - 26 Boş Yer

BİL110 - BİLGİSAYAR MÜHENDİSLİĞİNE GİRİŞ

Zorunlu

Kayıt Ol

Şube: A | 2 Kredi | 4 ECTS | T:2 U:0

Bilgisayar mühendisliği disiplininin genel tanıtımı ve temel kavramlarının öğretildiği derstir.

Şekil 2.15. Ders Seçim

Bu ekranda, öğrenci ders programına eklemek istediği dersleri ayrıntıları ile beraber görüntüleyebilir. Kayıt yaptırdığı derslerde çıkabilir. Kontenjan, gün-saat, dersi veren akademisyen gibi bilgileri görüntüleyebilir. Dolu kontenjan veya çakışma durumunda ilgili derse kaydı engellenir. Seçilen gün ve saat diliminde mevcut ders olup olmadığı kontrol edilir. Her günün kendi array'i alınır ve filtre kullanılarak aynı saat diliminde ders aranır. Mevcut ise ders ekleme engellenir.

Her ders kartında ilgili dersin silinebilmesi için buton bulunmaktadır. İlgili ders silindikten sonra backend çağrısı ile database üzerinde ilgili değişiklik yapılır.

Ders Programı Ekranı

Saat	Pazartesi	Salı	Çarşamba	Perşembe	Cuma
08:00					
09:00					
10:00			MAT151 MATEMATİKSEL ANALİZ I A118 - Teori		
11:00			MAT151 MATEMATİKSEL ANALİZ I A119 - Teori		
12:00					
13:00	BİL105 PROGRAMLAMA LABORATUVARI I LAB5 - Uygulama	FİZ103 MEKANİK LABORATUVARI LAB3 - Uygulama	MAT151 MATEMATİKSEL ANALİZ I A100 - Teori		
14:00	BİL105 PROGRAMLAMA LABORATUVARI I LAB1 - Uygulama	FİZ103 MEKANİK LABORATUVARI LAB4 - Uygulama	MAT151 MATEMATİKSEL ANALİZ I A101 - Teori		
15:00			MAT151 MATEMATİKSEL ANALİZ I LAB3 - Uygulama		
16:00					
17:00					

Kayıtlı Derslerim

BİL105 - PROGRAMLAMA LABORATUVARI I

Dersten Çık

Şube: A | 1 Kredi | 2 ECTS

Temel programlama kavramlarının uygulamalı olarak pekiştirildiği laboratuvar dersi.

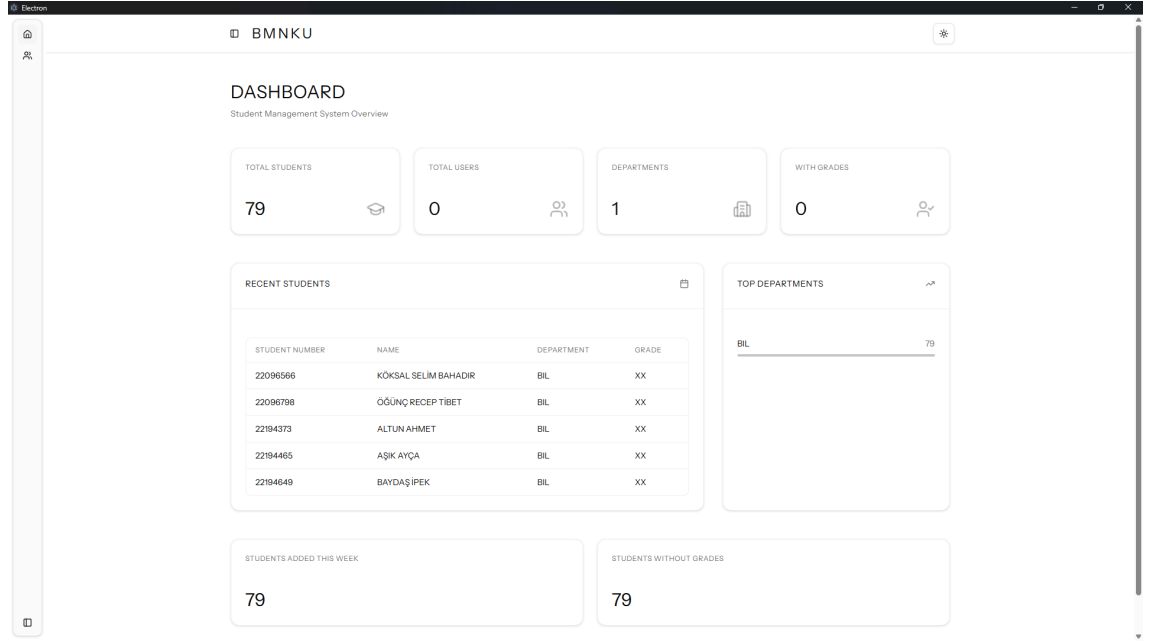
Pazartesi: 13:00-14:00 | LAB5 | Uygulama | Öğr. Gör. Can Şahin

Şekil 2.16. Ders Programı

Öğrencilerin haftalık ders programlarını görselleştiren ve yönetmelerini sağlayan tablo bazlı bir ekrandır. Backend üzerinden yapılan ilgili API çağrısı ile entegre çalışır. Haftanın 5 gününü içerecek şekilde dinamik bir tablo içerir.

Ekranın alt kısmında seçilen derslerin ayrıntılı bilgileri bulunur. Öğrenci silmek istediği dersi Dersten çık butonu ile programından silebilir. Ekranın üst kısmında seçilen derslerin toplam kredi ve Akts gibi bilgileri görüntülenir

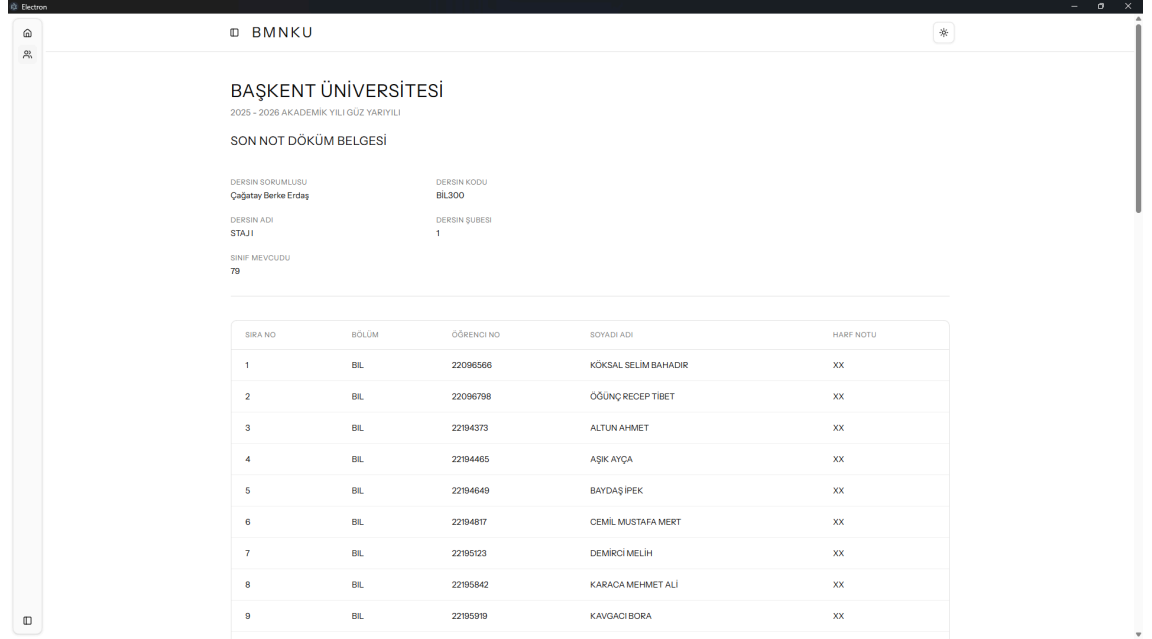
BMNKU Giriş / Başlangıç Ekranı



Şekil 3.1. Ana ekran

Bu ekran, Electron tabanlı masaüstü uygulamasının başlangıç ve yönlendirme ekranıdır. Kullanıcıya uygulamanın temel amacı olan ön koşul uygunluk kontrolü hakkında kısa bir bilgilendirme sunar. Danışman, bu ekran üzerinden öğrenci seçimi veya uygunluk sorgulama sürecine geçiş yapabilir. Basit ve sade tasarım sayesinde uygulamaya hızlı bir başlangıç yapılması hedeflenmiştir.

Öğrenci Seçim / Öğrenci Bilgisi Görüntüleme Ekranı



Electron BMNKU

BAŞKENT ÜNİVERSİTESİ

2025 - 2026 AKADEMİK YILI GÜZ YARIYILI

SON NOT DÖKÜM BELGESİ

DERSİN SORUMLUSU
Çağatay Berke Erdağ

DERSİN KODU
BIL300

DERSİN ADI
STAJ I

DERSİN ŞUBESİ
1

SINIF MEVCUDU
79

SIRA NO	BÖLÜM	ÖĞRENCİ NO	SOYADI ADI	HARF NOTU
1	BİL	22096566	KÖKSAL SELİM BAHADIR	XX
2	BİL	22096798	ÖĞÜNÇ RECEP TİBET	XX
3	BİL	22194373	ALTUN AHMET	XX
4	BİL	22194465	AŞIK AYÇA	XX
5	BİL	22194649	BAYDAŞ İPEK	XX
6	BİL	22194817	CEMİL MUSTAFA MERT	XX
7	BİL	22195123	DEMİRCİ MELİH	XX
8	BİL	22195642	KARACA MEHMET ALİ	XX
9	BİL	22195919	KAVGACI BORA	XX

Şekil 3.2. Öğrenci Görüntüleme

Bu ekranda akademik danışman, uygunluk kontrolü yapılacak öğrenciyi seçebilir veya öğrencinin ders geçmişi bilgilerini görüntüleyebilir. Öğrencinin daha önce aldığı dersler ve bu derslerin geçme/kalma durumları özet biçimde sunulmaktadır. Bu bilgiler, ön koşul değerlendirme sürecinin temel girdisini oluşturmaktadır. Ekran, danışmanın öğrencinin mevcut akademik durumunu hızlıca kavramasını sağlar.

Ders Ön Koşul Uygunluk Sorgulama Ekranı

PDF PROCESSING RESULTS

Student grades from uploaded PDF

Q Search by name, student number, or department...

STATUS:

All

DEPARTMENT:

All Departments

GENERATED FILES



STUDENT_REPORT.PDF



DOWNLOAD



STUDENT_SEPARATION.PDF



DOWNLOAD

SIRA NO	BÖLÜM	ÖĞRENCİ NO	SOYADI ADI	DURUM
1	BİL	22096566	KÖKSAL SELİM BAHADIR	Passed
2	BİL	22096798	ÖĞÜNÇ RECEP TİBET	Failed
3	BİL	22194373	ALTUN AHMET	Failed
4	BİL	22194465	AŞIK AYÇA	Failed
5	BİL	22194649	BAYDAŞ İPEK	Passed
6	BİL	22194817	CEMİL MUSTAFA MERT	Failed
7	BİL	22195123	DEMİRCİ MELİH	Passed
8	BİL	22195842	KARACA MEHMET ALİ	Failed
9	BİL	22195919	KAVGACI BORA	Passed

Şekil 3.3. Ders Ön Koşul Görüntüleme

Bu ekran, seçilen öğrenci için ders bazlı ön koşul uygunluk analizinin gerçekleştirildiği ana karar destek ekranıdır. Sistem, tanımlı ön koşul kurallarını kullanarak her ders için teknik değerlendirme sonucu olarak “Passed” (ön koşullar sağlandı) veya “Failed” (ön koşullar sağlanmadı) çıktısını üretir. Bu sonuçlar, danışman açısından sırasıyla “alabilir” ve “alamaz” akademik kararlarına karşılık gelmektedir. Değerlendirme süreci kullanıcı etkileşimi sonrasında anlık olarak gerçekleştirilerek manuel transkript inceleme ihtiyacını ortadan kaldırır.

PDF Yükleme / Doküman Alma Ekranı



Şekil 3.4. Pdf Yükleme

Bu ekran, akademik danışmanın öğrencinin ders geçmişine ait PDF dokümanını (transkript veya benzeri akademik belge) sisteme yükleyebildiği giriş ekranıdır. Kullanıcı, sürükle-bırak yöntemiyle ilgili PDF dosyasını kolayca uygulamaya aktarabilir. Yüklenen doküman, ön koşul uygunluk değerlendirmesinde kullanılacak verilerin elde edilmesi için temel girdi olarak değerlendirilir. Bu yaklaşım, manuel veri girişini azaltarak danışmanlık sürecinin daha hızlı ve hatasız yürütülmesini amaçlamaktadır.

5.KAYNAKÇA

- [1]1. Microsoft. (2024). ASP.NET Core documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/>
- [2]3. Smith, J., & Johnson, P. (2023). Entity Framework Core in Action: Manning Publications approach to object-relational mapping in .NET applications. *Journal of Software Engineering and Applications*, 16(4), 112-128. <https://doi.org/10.4236/jsea.2023.164007>
- [3]6. Microsoft. (2024). Entity Framework Core documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/ef/core/>
- [4]Duckett, J. (2014). *JavaScript and JQuery: Interactive front-end web development*. Wiley.
- [5]Banks, A., & Porcello, E. (2020). *Learning React: Modern patterns for developing React apps* (2. baskı). O'Reilly Media.
- [6]Electron. (n.d.). Introduction. Retrieved January 3, 2026, from <https://electronjs.org/docs/latest>
- [7]Electron. (n.d.). Building your first app. Retrieved January 3, 2026, from <https://electronjs.org/docs/latest/tutorial/tutorial-first-app>
- [8]Electron. (n.d.). Packaging your application. Retrieved January 3, 2026, from <https://electronjs.org/docs/latest/tutorial/tutorial-packaging>